



400-650-7088

TongESB

V7. 1. 3. 0

EDI 操作指导

版本：7130A01

北京东方通科技股份有限公司

2025 年 08 月

声明

版权

Copyright ©2025北京东方通科技股份有限公司。保留所有权利。

版权声明

- 本文档的所有权和知识产权归属于北京东方通科技股份有限公司（以下简称“东方通”）或其关联公司所有。
- 除非另有明确约定，东方通不授予任何明示或暗示的许可或权利，使用者不得以任何方式将本文档中的任何内容用于商业目的。
- 东方通对本文档享有版权。本文档的所有内容，包括但不限于文字、图像、图表、图标、示意图、屏幕截图等，均受版权法和国际版权条约的保护。
- 未经东方通明确授权，任何人不得对本文档以任何形式复制、修改、传播、分发、展示或进行衍生创作。

商标声明

- 、TongTech 标识以及其他相关东方通图形、徽标、服务名称和商标（以下统称为“东方通商标”）是东方通或其关联公司的注册商标或商标。
- 未经东方通明确授权，任何人不得以任何方式使用、复制或展示东方通商标。此外，未经东方通事先书面同意，不得将东方通商标与其他商标、标识或徽标进行混淆、链接或结合使用。
- 除非另有明确约定，本商标声明不授予任何明示或暗示的许可或权利，对东方通商标的使用须获得东方通的明确授权。
- 本文档提及的所有其他商标、标识、徽标或产品名称均为其各自所有者的财产，并可能受其各自的商标法保护。

免责声明

请在使用东方通产品之前仔细阅读本免责声明，并根据自身情况判断是否继续使用。如有任何问题，请联系我们的客户支持团队。

- 本产品的使用是基于用户自己的判断和风险评估。本文档仅作为使用指导，不对使用本产品所产生的结果做任何明示或暗示的担保。
- 东方通不对用户未按照本文档中的指导正确使用东方通产品而导致的任何损失或损害承担责任；
- 本文档可能会包含第三方提供的内容、链接或资源，这些内容由第三方自行负责。东方通对于这些内容的准确性、完整性、合法性或可靠性不承担责任。用户在使用这些内容时应自行判断和承担风险。
- 由于产品版本升级或其他原因，本文档内容会不定期更新。东方通保留在不事先通知用户的情况下随时对文档进行修改、更新或中止的权利。

如需获取有关东方通产品的许可或使用权，请联系北京东方通科技股份有限公司或授权代理商。任何违反本声明的行为将受到适用法律的追究。

版本变更说明

手册版本	适用产品	更新内容	更新日期
V7130A01	7.1.3.0	无	2025/08/22
V7120A01	7.1.2.0	无	2025/05/20
V7120A01	7.1.0.0	新增 EDI 详细操作指导。	2024/07/30

前言

本文档是产品的用户使用手册之一。

阅读前注意事项

通过阅读本文档，您确认并同意自行承担因未具备必要专业背景 and 知识而导致的任何风险或后果。在使用本文档中提供的信息和指南时，请始终谨慎，并在必要时寻求专业人士建议和指导。

- 适用对象

本文档主要适用于使用本产品的系统管理员阅读，部分内容同样适用于基于本产品进行应用开发或应用部署的人员阅读。

- 专业背景

本文内容可能涉及到操作系统、服务器硬件、网络等相关领域的知识。请确保您具备相关背景 and 知识，以便更好地理解 and 应用本手册的内容。

- 技能要求

为了能够充分理解 and 应用本文档的内容，建议您具备如下技能：

- 掌握 Linux 系统基本操作；
- 熟悉数据库（MySQL 等）。
- 安装产品。

- 术语和概念

本文档可能使用一些专业术语和概念。请确保您熟悉这些术语和概念，或者有能力查阅相关资料以便进一步理解。

- 实践经验

为了最大程度地受益于本文档，建议您具备一定的实践经验。这将帮助您更好地应用文档中的操作指南 and 建议。

请注意，本文档不适用于没有相关专业背景 and 知识的用户。

若您对本文档的内容 or 所需背景有任何疑问，请在使用之前咨询相关专业人士 or 寻求额外的支持。

用户手册集

为您提供的用户使用手册集包含如下文档：

编号	手册名称	说明
1	001_TongESB_V7.1.3.0 使用手册	提供TongESB安装及使用说明。
2	002_TongESB_V7.1.3.0 数据库服务化操作指导	详细介绍数据库服务化中创建、配置、部署、调用数据服务的详细操作。

编号	手册名称	说明
3	003_TongESB_V7.1.3.0 数据映射操作指导	详细介绍数据映射的详细操作。
4	004_TongESB_V7.1.3.0 EDI操作指导	详细介绍EDI的详细操作。
5	005_TongESB_V7.1.3.0 动态路由语法规则手册	提供API动态路由语法规则说明。
6	006_TongESB_V7.1.3.0 链路跟踪	提供链路跟踪的安装及配置。
7	007_TongESB_V7.1.3.0 安全加固配置说明	提供安全加固配置说明。
8	008_TongESB_V7.1.3.0 常见问题	提供常见问题的解决方案。

技术支持

东方通产品将为您提供全方位的技术支持，您可以通过以下方式获得技术支持：

- 网址：www.tongtech.com
- 电话：400-650-7088
- 邮箱：support@tongtech.com

您在取得技术支持时，请提供如下信息：

- 您的姓名
- 您的公司信息
- 您的联系方式
- 操作系统及其版本
- 产品版本号
- 日志等错误的详细信息

目录

声明	1
版本变更说明	2
前言	3
1. 功能简介	6
1.1. 配置说明	6
1.1.1. EDI	6
1.1.2. EDIFACT	7
1.1.3. 命名空间	7
1.1.4. 导出结果类型	7
2. edi	9
2.1. EDI DFDL Schema	12
2.2. DFDL 简要说明	13
2.2.1. 区段声明	13
2.2.2. 区段组	14
2.2.3. 区段数据元素	15
2.2.4. 复合数据元素	16
2.2.5. 导入	17
2.2.6. 支持类型	18
3. edifact	19
3.1. schemas	19
3.2. 消息类型	21
3.3. 结构	30
3.4. 自定义脚本	33

1. 功能简介

EDI 组件执行过程中会消费数据源格式为 XML 或 EDI 的源文件，消费过程中将通过源数据生成事件流，并通过事件流触发相应的访问者（如 Java、DFDL 等）。

这样处理的目的是为了生成不同格式的结果流便于数据传输（如绑定源数据到 POJO、Java 图谱以及多种数据片段等）。

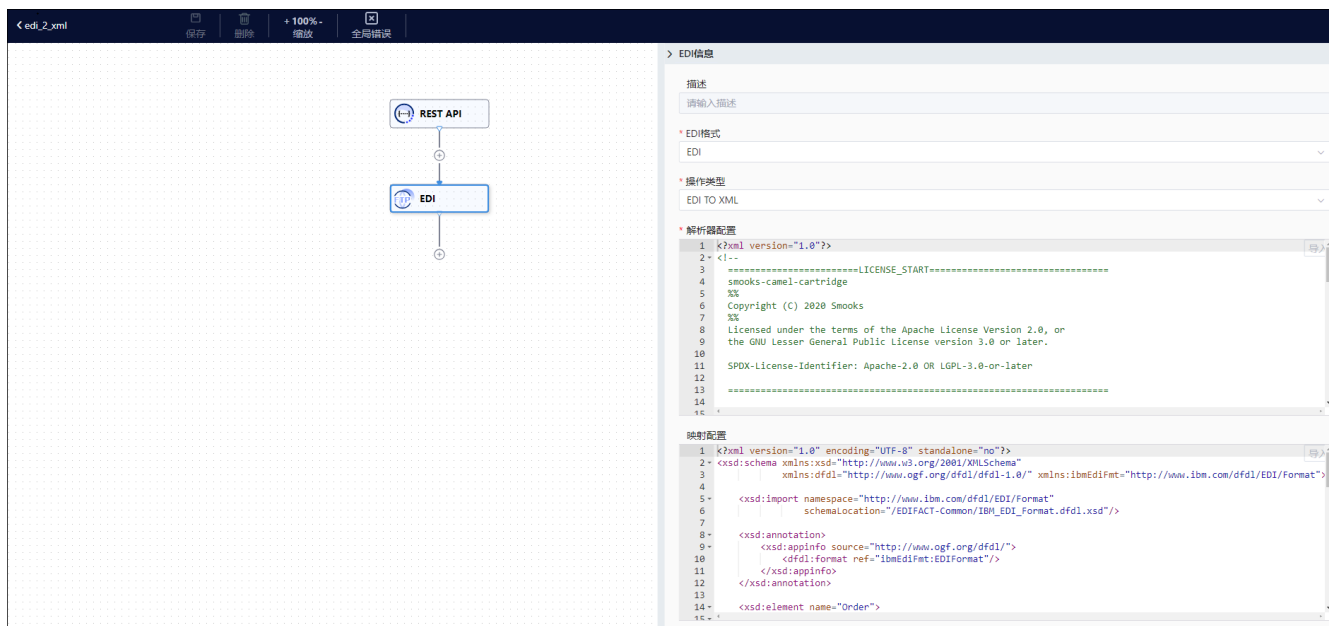
EDI 组件主要还是通过消费源文件进行 EDI 文档的解析和生成。

1.1. 配置说明

EDI 组件使用过程需要用户提供相应的解析器配置和映射配置，其中 EDIFACT 会根据用户选择进行自动生成相应配置。

1.1.1. EDI

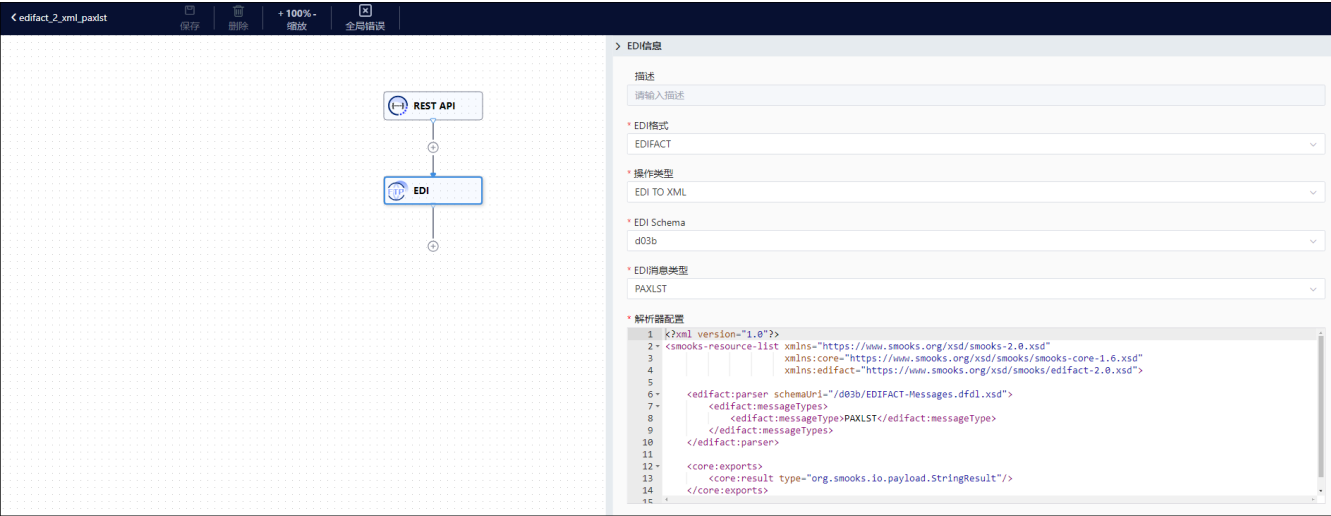
关于 edi 文档的解析/生成，如下图所示。



配置属性说明，如下表所示。

配置属性名	可选值	默认值	必需
EDI 格式	EDI、EDIFACT	EDIFACT	是
操作类型	EDI TO XML、XML TO EDI	EDI TO XML	是
解析器配置	无	无	是
映射配置	无	无	是

1.1.2. EDIFACT



配置属性说明，如下表所示。

配置属性名	可选值	默认值	必需
EDI 格式	EDI、EDIFACT	EDIFACT	是
操作类型	EDI TO XML、XML TO EDI	EDI TO XML	是
EDI Schema	UN EDIFACT 官方提供，逐年更行	d00a	是
EDI 消息类型	UN EDIFACT 官方提供	APERAK	是
解析器配置	无	无	是

1.1.3. 命名空间

名称	地址
smooks-core-1.6	https://www.smooks.org/xsd/smooks/smooks-core-1.6.xsd
smooks-2.0	https://www.smooks.org/xsd/smooks-2.0.xsd
IBM_EDI_Format	http://www.ibm.com/dfdl/EDI/Format
dfdl	http://www.ogf.org/dfdl/dfdl-1.0/
dfdl-edi	http://www.ibm.com/dfdl/EDI/Format
edi	https://www.smooks.org/xsd/smooks/edi-2.0.xsd

1.1.4. 导出结果类型

类型	说明
javax.xml.transform.stream.StreamResult	Stream
javax.xml.transform.dom.DOMResult	DOM
org.smooks.io.payload.JavaResult	JAVA
org.smooks.io.payload.StringResult	String

解析器配置中通过 core:exports 配置导出类型，导出配置示例，如下所示。

```
<smooks-resource-list xmlns="https://www.smooks.org/xsd/smooks-2.0.xsd"
xmlns:core="https://www.smooks.org/xsd/smooks/smooks-core-1.6.xsd">
  <core:exports>
    <core:result type="org.smooks.io.payload.JavaResult"/>
  </core:exports>
</smooks-resource-list>
```

2. edi

EDI 组件解析 edi 文档并将其序列化，或者解析 DFDL 命名语法定义的文档生成事件流和 edi 文档。

dfdl 支持反序列化非 XML 数据以及根据 [DFDL schema](#) 序列化 XML 记录。

edi 文档的解析和生成，需要用户进行编写解析器配置和映射配置，其中解析器配置较简单，配置 parser 或者 unparser 即可进行解析或生成 edi 文档，映射配置主要依赖于 dfdl schema，可以参考 [DFDL schema](#) 官方文档。

以 edi 解析为例，dfdl 配置，如下所示。

```
<?xml version="0" encoding="UTF-8" standalone="no"?>
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  xmlns:dfdl="http://www.wogfor.com/dfdl/dfdl-0/" xmlns:ibmEdiFmt="http://www.ibm.com/dfdl/EDI/Format">

  <xsd:import namespace="http://www.ibm.com/dfdl/EDI/Format"
    schemaLocation="/EDIFACT-Common/IBM_EDIFormatdfdlxsd"/>

  <xsd:annotation>
    <xsd:appinfo source="http://www.wogfor.com/dfdl/">
      <dfdl:format ref="ibmEdiFmt:EDIFormat"/>
    </xsd:appinfo>
  </xsd:annotation>

  <xsd:element name="Order">
    <xsd:complexType>
      <xsd:sequence>
        <xsd:sequence dfdl:initiatedContent="yes">
          <xsd:element dfdl:initiator="HDR" dfdl:ref="ibmEdiFmt:EDISegmentFormat" name="header">
            <xsd:complexType>
              <xsd:sequence dfdl:ref="ibmEdiFmt:EDISegmentSequenceFormat">
                <xsd:element name="order-id" type="xsd:string"/>
                <xsd:element name="status-code" type="xsd:string"/>
                <xsd:element name="net-amount" type="xsd:string"/>
                <xsd:element name="total-amount" type="xsd:string"/>
                <xsd:element name="tax" type="xsd:string"/>
                <xsd:element name="date" type="xsd:string"/>
              </xsd:sequence>
            </xsd:complexType>
          </xsd:element>
          <xsd:element dfdl:initiator="CUS" dfdl:ref="ibmEdiFmt:EDISegmentFormat" name="customer-
details">
            <xsd:complexType>
              <xsd:sequence dfdl:ref="ibmEdiFmt:EDISegmentSequenceFormat">
```

```

<xsd:element name="username" type="xsd:string"/>
<xsd:element name="name">
  <xsd:complexType>
    <xsd:sequence dfdl:ref="ibmEdiFmt:EDICompositeSequenceFormat">
      <xsd:element name="firstname" type="xsd:string"/>
      <xsd:element name="lastname" type="xsd:string"/>
    </xsd:sequence>
  </xsd:complexType>
</xsd:element>
<xsd:element name="state" type="xsd:string"/>
</xsd:sequence>
</xsd:complexType>
</xsd:element>
<xsd:element dfdl:initiator="ORD" dfdl:ref="ibmEdiFmt:EDISegmentFormat" name="order-item"
maxOccurs="unbounded">
  <xsd:complexType>
    <xsd:sequence dfdl:ref="ibmEdiFmt:EDISegmentSequenceFormat">
      <xsd:element name="position" type="xsd:string"/>
      <xsd:element name="quantity" type="xsd:string"/>
      <xsd:element name="product-id" type="xsd:string"/>
      <xsd:element name="title" type="xsd:string"/>
      <xsd:element name="price" type="xsd:string"/>
    </xsd:sequence>
  </xsd:complexType>
</xsd:element>
</xsd:sequence>
</xsd:sequence>
</xsd:complexType>
</xsd:element>
</xsd:schema>

```

输入 edi 数据，如下所示。

```

HDR*1*0*59.97*64.92*4.95*Wed Nov 15 13:45:28 EST 2006
CUS*user1*Harry^Fletcher*SD
ORD*1*1*364*The 40-Year-Old Virgin*29.98
ORD*2*1*299*Pulp Fiction*29.99

```

解析配置，如下所示。

```

<?xml version="1.0"?>

<smooks-resource-list
  xmlns="https://www.smooks.org/xsd/smooks-2.0.xsd"
  xmlns:edi="https://www.smooks.org/xsd/smooks/edi-2.0.xsd"

```

```

xmlns:core="https://www.smooks.org/xsd/smooks/smooks-core-1.6.xsd">

<!--
Configure the EDI Reader to process the message stream into a stream of SAX events
-->
<edi:parser schemaUri="/edi-mapping-model.dfdl.xsd" segmentTerminator="%NL;" dataElementSeparator="*"
    compositeDataElementSeparator="^"/>

<core:exports>
    <core:result type="javax.xml.transform.dom.DOMResult"/>
</core:exports>

</smooks-resource-list>

```

- schemaUri: 关于 edi 文档解析/生成的 DFDL schema 描述。
- segmentTerminator: 相关数据的结束符, 如 %NL 在 DFDL 实例中将被视为换行。默认值"%NL;%WSP*;%WSP*;"。
- dataElementSeparator: 数据元素分隔符, 默认值"+"。
- compositeDataElementSeparator: 复合数据元素的分隔符, 默认值":"。
- validationMode: 校验模式。

模式	说明
Off	关闭 DFDL 的模式校验。
Limited	执行 XSD 校验, 如 facets、minLength、maxLength、enumeration、minInclusive、minExclusive、maxInclusive、maxExclusive、maxOccurs; 校验失败将打印失败日志在日志中, 但不会中断解析或取消解析。
Full	使用 Xerces 执行完整的schema验证。校验失败将取消解析同时抛出异常。

- escapeCharacter: 默认值"?"。
- repetitionSeparator: 默认值"*"。
- decimalSign: 默认值"."。
- triadSeparator: 默认值","。

输出结果, 如下所示。

```

<Order>
  <header>
    <order-id>1</order-id>
    <status-code>0</status-code>
    <net-amount>59.97</net-amount>
    <total-amount>64.92</total-amount>

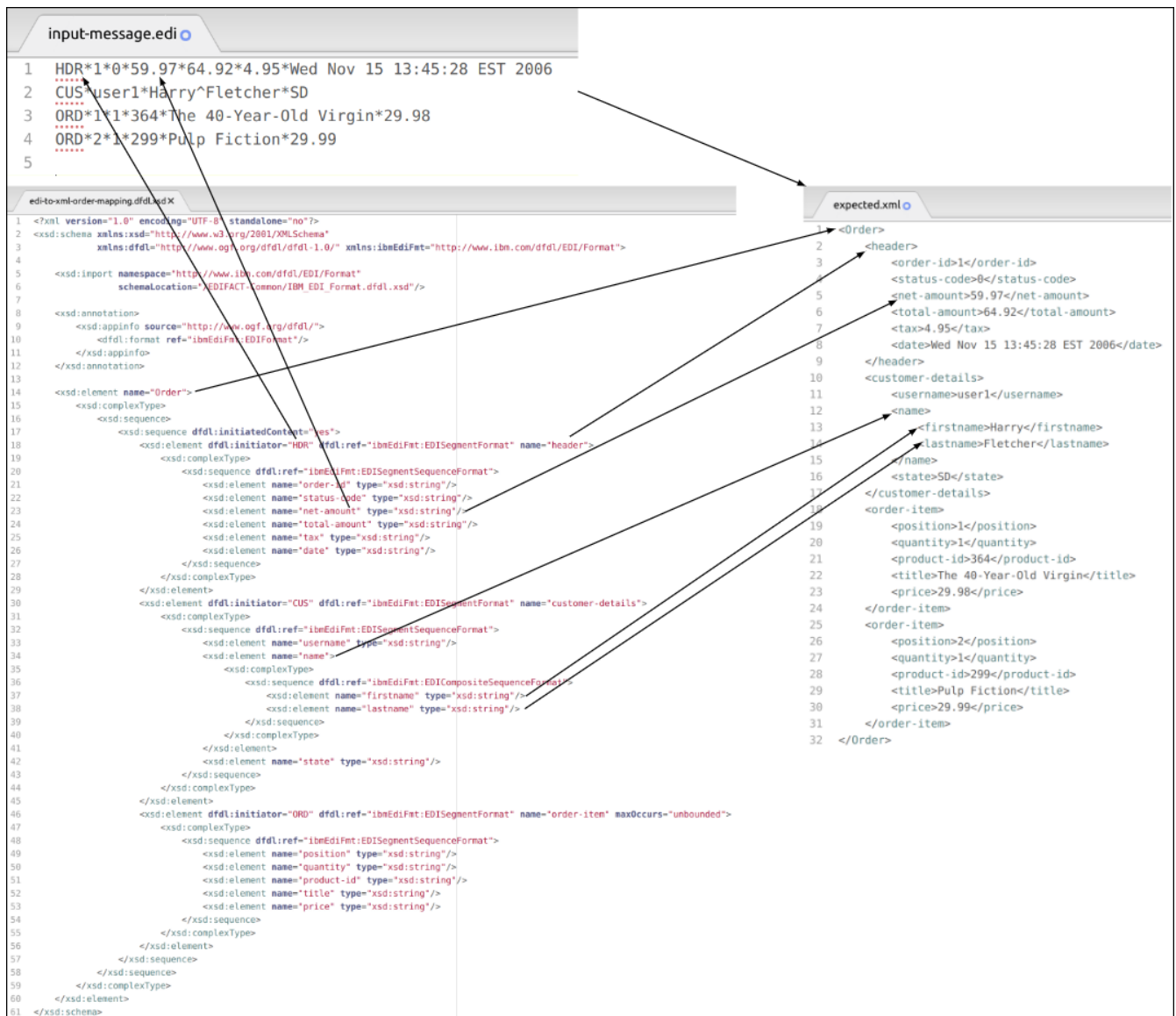
```

```
<tax>4.95</tax>
<date>Wed Nov 15 13:45:28 EST 2006</date>
</header>
<customer-details>
  <username>user1</username>
  <name>
    <firstname>Harry</firstname>
    <lastname>Fletcher</lastname>
  </name>
  <state>SD</state>
</customer-details>
<order-item>
  <position>1</position>
  <quantity>1</quantity>
  <product-id>364</product-id>
  <title>The 40-Year-Old Virgin</title>
  <price>29.98</price>
</order-item>
<order-item>
  <position>2</position>
  <quantity>1</quantity>
  <product-id>299</product-id>
  <title>Pulp Fiction</title>
  <price>29.99</price>
</order-item>
</Order>
```

2.1. EDI DFDL Schema

用户自定义的 DFDL schema 支持 edi 文档的解析和生成；在使用过程中需要导入 IBM_EDI_Format.dfdl.xsd DFDL schema，该 schema 中定义了公共的 edi 结构（如数据段和数据元素）。

输入数据、DFDL schema、输出结果映射关系，如下图所示。



- Input-message.edi: 输入/输出的 edi 文档。
- edi-to-xml-order.mapping.dfdl.xsd: 描述了edi 文档的解析/生成的映射规则。
- Expected.xml: 输出的结果。

2.2. DFDL 简要说明

2.2.1. 区段声明

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
```

```
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema"
```

```
xmlns:dfdl="http://www.ogf.org/dfdl/dfdl-1.0/"
```

```
xmlns:ibmEdiFmt="http://www.ibm.com/dfdl/EDI/Format">
```

```
<xsd:import namespace="http://www.ibm.com/dfdl/EDI/Format" schemaLocation="/EDIFACT-Common/IBM_EDI_Format.dfdl.xsd"/>
```

```

<xsd:annotation>
  <xsd:appinfo source="http://www.ogf.org/dfdl/">
    <dfdl:format ref="ibmEdiFmt:EDIFormat"/>
  </xsd:appinfo>
</xsd:annotation>

<xsd:element dfdl:initiator="HDR" // (1)
  name="header" // (2)
  dfdl:ref="ibmEdiFmt:EDISegmentFormat"> // (3)
  <xsd:complexType>
    ..
  </xsd:complexType>
</xsd:element>
</xsd:schema>

```

- dfdl:initiator: 定义区段代码分辨符。
- name: 定义指定XML段的映射键名。
- dfdl:ref: 引用相关区段的定义，如 ibmEdiFmt:EDISegmentFormat 包含了 edi 文档的区段结构定义。

区段定义中 minOccurs 和 maxOccurs 属性为可选属性，默认值均为 1。

minOccurs 和 maxOccurs 属性通常用于控制区段的可选和必需特征。

当 maxOccurs 值为 unbounded 时，代表区段在 edi 文档中可以重复任意次。

2.2.2. 区段组

当区段定义的 maxOccurs 属性设置了多个以及在区段定义中添加其它区段定义时，将隐式创建区段组。

如下 HDR 就是一个区段组，因为 maxOccurs 值为 unbounded，以及在 HDR 区段定义中好包含 CUS 和 ORD 区段定义。

```

<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  xmlns:dfdl="http://www.ogf.org/dfdl/dfdl-1.0/"
  xmlns:ibmEdiFmt="http://www.ibm.com/dfdl/EDI/Format">

  <xsd:import namespace="http://www.ibm.com/dfdl/EDI/Format" schemaLocation="/EDIFACT-
Common/IBM_EDI_Format.dfdl.xsd"/>

  <xsd:annotation>
    <xsd:appinfo source="http://www.ogf.org/dfdl/">
      <dfdl:format ref="ibmEdiFmt:EDIFormat"/>
    </xsd:appinfo>
  </xsd:annotation>

```

```

<xsd:element dfdl:initiator="HDR" name="order" maxOccurs="unbounded">
  <xsd:complexType>
    <xsd:sequence>
      <xsd:sequence dfdl:ref="ibmEdiFmt:EDISegmentFormat">
        ..
      </xsd:sequence>
      <xsd:element dfdl:initiator="CUS" dfdl:ref="ibmEdiFmt:EDISegmentFormat" name="customer-details">
        <xsd:complexType>
          ..
        </xsd:complexType>
      </xsd:element>
      <xsd:element dfdl:initiator="ORD" dfdl:ref="ibmEdiFmt:EDISegmentFormat" name="order-item"
        maxOccurs="unbounded">
        <xsd:complexType>
          ..
        </xsd:complexType>
      </xsd:element>
    </xsd:sequence>
  </xsd:complexType>
</xsd:element>
</xsd:schema>

```

2.2.3. 区段数据元素

区段数据元素是 DFDL 序列元素中的子元素，通过 dfdl:ref 指定相应值进行引用，如 ibmEdiFmt:EDISegmentSequenceFormat。

```

<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  xmlns:dfdl="http://www.ogf.org/dfdl/dfdl-1.0/"
  xmlns:ibmEdiFmt="http://www.ibm.com/dfdl/EDI/Format">

  <xsd:import namespace="http://www.ibm.com/dfdl/EDI/Format" schemaLocation="/EDIFACT-
Common/IBM_EDI_Format.dfdl.xsd"/>

  <xsd:annotation>
    <xsd:appinfo source="http://www.ogf.org/dfdl/">
      <dfdl:format ref="ibmEdiFmt:EDIFormat"/>
    </xsd:appinfo>
  </xsd:annotation>

  <xsd:element dfdl:initiator="HDR" dfdl:ref="ibmEdiFmt:EDISegmentFormat" name="header">
    <xsd:complexType>
      <xsd:sequence dfdl:ref="ibmEdiFmt:EDISegmentSequenceFormat">

```



```

    <xsd:element name="order-id" type="xsd:string"/>
    <xsd:element name="status-code" type="xsd:string"/>
    <xsd:element name="net-amount" type="xsd:string"/>
    <xsd:element name="total-amount" type="xsd:string"/>
    <xsd:element name="tax" type="xsd:string"/>
    <xsd:element name="date" type="xsd:string"/>
  </xsd:sequence>
</xsd:complexType>
</xsd:element>
</xsd:schema>

```

xsd:sequence 中的每个 xsd:element 子节点都代表一个edi数据元素。name 属性值是在目标 XML 元素要捕获的值。

2.2.4. 复合数据元素

若 xsd:sequence 中包含 xsd:sequence 元素，且其 dfdl:ref 值为 ibmEdiFmt:EDICompositeSequenceFormat 则表示复合数据元素。

```

<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  xmlns:dfdl="http://www.ogf.org/dfdl/dfdl-1.0/"
  xmlns:ibmEdiFmt="http://www.ibm.com/dfdl/EDI/Format">

  <xsd:import namespace="http://www.ibm.com/dfdl/EDI/Format" schemaLocation="/EDIFACT-
Common/IBM_EDI_Format.dfdl.xsd"/>

  <xsd:annotation>
    <xsd:appinfo source="http://www.ogf.org/dfdl/">
      <dfdl:format ref="ibmEdiFmt:EDIFormat"/>
    </xsd:appinfo>
  </xsd:annotation>

  <xsd:element dfdl:initiator="CUS" dfdl:ref="ibmEdiFmt:EDISegmentFormat" name="customer-details">
    <xsd:complexType>
      <xsd:sequence dfdl:ref="ibmEdiFmt:EDISegmentSequenceFormat">
        <xsd:element name="username" type="xsd:string"/>
        <xsd:element name="name">
          <xsd:complexType>
            <xsd:sequence dfdl:ref="ibmEdiFmt:EDICompositeSequenceFormat">
              <xsd:element name="firstname" type="xsd:string"/>
              <xsd:element name="lastname" type="xsd:string"/>
            </xsd:sequence>
          </xsd:complexType>
        </xsd:element>
      </xsd:sequence>
    </xsd:complexType>
  </xsd:element>

```

```

        </xsd:element>
        <xsd:element name="state" type="xsd:string"/>
    </xsd:sequence>
</xsd:complexType>
</xsd:element>
</xsd:schema>

```

2.2.5. 导入

多数 edi 消息使用相同的区段定义，可以在 dfdl 配置的顶层导入避免重复配置。

```

<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  xmlns:dfdl="http://www.ogf.org/dfdl/dfdl-1.0/"
  xmlns:ibmEdiFmt="http://www.ibm.com/dfdl/EDI/Format"
  xmlns:def="def">

  <xsd:import namespace="http://www.ibm.com/dfdl/EDI/Format" schemaLocation="/EDIFACT-
Common/IBM_EDI_Format.dfdl.xsd"/>
  <xsd:import namespace="def" schemaLocation="example/edi-segment-definition.xml"/>

  <xsd:annotation>
    <xsd:appinfo source="http://www.ogf.org/dfdl/">
      <dfdl:format ref="ibmEdiFmt:EDIFormat"/>
    </xsd:appinfo>
  </xsd:annotation>

  <xsd:element name="Order">
    <xsd:complexType>
      <xsd:sequence>
        <xsd:sequence dfdl:initiatedContent="yes">
          <xsd:element dfdl:initiator="HDR" dfdl:ref="ibmEdiFmt:EDISegmentFormat" name="header"
type="def:HDR"/>
          <xsd:element dfdl:initiator="CUS" dfdl:ref="ibmEdiFmt:EDISegmentFormat" name="customer-details"
type="def:CUS"/>
          <xsd:element dfdl:initiator="ORD" dfdl:ref="ibmEdiFmt:EDISegmentFormat" name="order-item"
maxOccurs="unbounded" type="def:ORD"/>
        </xsd:sequence>
      </xsd:sequence>
    </xsd:complexType>
  </xsd:element>
</xsd:schema>

```

上述配置中通过 xsd:import 元素导入了 IBM_EDI_Format。

2.2.6. 支持类型

xsd:element 中 type 属性可以指定数据类型同时进行类型校验。

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  xmlns:dfdl="http://www.ogf.org/dfdl/dfdl-1.0/"
  xmlns:ibmEdiFmt="http://www.ibm.com/dfdl/EDI/Format">

  <xsd:import namespace="http://www.ibm.com/dfdl/EDI/Format" schemaLocation="/EDIFACT-
Common/IBM_EDI_Format.dfdl.xsd"/>

  <xsd:annotation>
    <xsd:appinfo source="http://www.ogf.org/dfdl/">
      <dfdl:format ref="ibmEdiFmt:EDIFormat"/>
    </xsd:appinfo>
  </xsd:annotation>

  <xsd:element dfdl:initiator="HDR" dfdl:ref="ibmEdiFmt:EDISegmentFormat" name="header">
    <xsd:complexType>
      <xsd:sequence dfdl:ref="ibmEdiFmt:EDISegmentSequenceFormat">
        <xsd:element name="order-id" type="xsd:string"/>
        <xsd:element name="status-code" type="xsd:int" dfdl:textNumberPattern="0"/>
        <xsd:element name="net-amount" type="xsd:decimal" dfdl:textNumberPattern="0"/>
        <xsd:element name="total-amount" type="xsd:decimal" dfdl:textNumberPattern="#.#"/>
        <xsd:element name="tax" type="xsd:decimal" dfdl:textNumberPattern="#.#"/>
        <xsd:element name="date" type="xsd:date"/>
      </xsd:sequence>
    </xsd:complexType>
  </xsd:element>
</xsd:schema>
```

3. edifact

EDI组件通过 UN EDIFACT 官方消息定义的 zip 预生成了 EDI DFDL schema，以便支持EDIFACT 文档的解析和生成。针对 EDIFACT 文档的解析和生成，用户无需编写解析器配置和映射配置，用户选择操作类型、EDI Schema 和 消息类型后，程序会自动生成相应配置。

EDIFACT 相关配置和流程与 EDI 相似，这里不做详细说明，细节可参考 EDI 一节。本文简单介绍 EDIFACT 用的 schema 和 消息类型。

3.1. schemas

UN EDIFACT 每年按照 A、B 两个版本的接着逐年发布 EDIFACT 的消息定义规范。

下表为 EDI 组件所支持的版本。

名称	说明
d93a	1993 年版本 a
d94a	1994 年版本 a
d94b	1994 年版本 b
d95a	1995 年版本 a
d95b	1995 年版本 b
d96a	1996 年版本 a
d96b	1996 年版本 b
d97a	1997 年版本 a
d97b	1997 年版本 b
d98a	1998 年版本 a
d98b	1998 年版本 b
d99a	1999 年版本 a
d99b	1999 年版本 b
d00a	2000 年版本 a
d00b	2000 年版本 b
d01a	2001 年版本 a

名称	说明
d01b	2001 年版本 b
d01c	2001 年版本 c
d02a	2002 年版本 a
d02b	2002 年版本 b
d03a	2003 年版本 a
d03b	2003 年版本 b
d04a	2004 年版本 a
d04b	2004 年版本 b
d05a	2005 年版本 a
d05b	2005 年版本 b
d06a	2006 年版本 a
d06b	2006 年版本 b
d07a	2007 年版本 a
d07b	2007 年版本 b
d08a	2008 年版本 a
d08b	2008 年版本 b
d09a	2009 年版本 a
d09b	2009 年版本 b
d10a	2010 年版本 a
d10b	2010 年版本 b
d11a	2011 年版本 a
d11b	2011 年版本 b
d12a	2012 年版本 a
d12b	2012 年版本 b

名称	说明
d13a	2013 年版本 a
d13b	2013 年版本 b
d14a	2014 年版本 a
d14b	2014 年版本 b
d15a	2015 年版本 a
d15b	2015 年版本 b
d16a	2016 年版本 a
d16b	2016 年版本 b
d17a	2017 年版本 a
d17b	2017 年版本 b
d18a	2018 年版本 a
d18b	2018 年版本 b
d19a	2019 年版本 a
d19b	2019 年版本 b
d20a	2020 年版本 a
d20b	2020 年版本 b
d21a	2021 年版本 a
d21b	2021 年版本 b

3.2. 消息类型

下表为 EDI 组件所支持的消息类型。

名称	说明
APERAK	应用错误与确认报文
AUTACK	确认报文
AUTHOR	授权报文

名称	说明
AVLREQ	可用性请求
AVLRSP	可用性应答
BALANC	资产负债报文
BANSTA	银行业务状态报文
BAPLIE	船图-积载图报文
BERMAN	泊位管理报文
BMISRM	散装船舶检查总结报告报文
BOPBNK	银行交易及有价证券交易报告报文
BOPCUS	余额支付客户交易报告报文
BOPDIR	直接付款余额申报报文
BOPINF	客户余额支付信息报文
BUSCRD	商业信用报告报文
CALINF	船舶挂靠信息报文
CASINT	民事法律行政诉讼请求报文
CASRES	民事诉讼回复报文
CHACCO	会计科目表报文
CLASET	分类信息集报文
CNTCND	合同条款报文
COACSU	商业帐单汇总报文
COARRI	集装箱装/卸报告报文
CODECO	集装箱进/出门报告报文
CODENO	许可证到期/开始结关通知报文
COEDOR	运输设备堆存和配置报告报文
COHAOR	集装箱特殊装卸指示报文

名称	说明
COLREQ	跟单托收请求报文
COMDIS	商业纠纷报文
CONAPW	未决工程建议报文
CONDPV	直接支付估价报文
CONDRA	绘图管理报文
CONDRO	绘图机构报文
CONEST	合同订立报文
CONITT	招标报文
CONPVA	支付估价报文
CONQVA	质量评估报文
CONRPW	开工前回复报文
CONTEN	投标报文
CONWQD	工作项目量确定报文
COPARN	集装箱通知报文
COPAYM	付款支出
COPINO	集装箱预通知报文
COPRAR	集装箱装/卸指示报文
COREOR	集装箱放行指示报文
COSTCO	装箱/拆箱确认报文
COSTOR	装箱/拆箱指示报文
CREADV	贷记通知报文
CREEXT	扩展贷记通知报文
CREMUL	多重贷记通知报文
CUSCAR	海关货物报告报文

名称	说明
CUSDEC	海关申报报文
CUSEXP	海关快件货物申报报文
CUSPED	定期报关报文
CUSREP	海关运输工具报告报文
CUSRES	海关回执报文
DEBADV	借记通知报文
DEBMUL	多重借记通知报文
DEBREC	债务恢复报文
DELFOR	交货计划报文
DELJIT	适时交货报文
DESADV	发货通知报文
DESTIM	设备损坏和维修预算报文
DGRECA	危险品概况报文
DIRDEB	直接借记报文
DIRDEF	目录定义报文
DMRDEF	数据维护请求定义报文
DMSTAT	数据维护状态报告/查询报文
DOCADV	跟单信用证通知报文
DOCAMA	跟单信用证更改通知报文
DOCAMI	跟单信用证更改信息报文
DOCAMR	跟单信用证更改请求报文
DOCAPP	跟单信用证申请报文
DOCARE	跟单信用证更改回复报文
DOCINF	跟单信用证签发信息报文

名称	说明
ENTREC	会计条目报文
FINCAN	金融撤销报文
FINPAY	多重银行间资金转移报文
FINSTA	财务报表报文
GENRAL	一般用途报文
GESMES	一般统计报文
GOVCBR	政府跨境监管报文
HANMOV	货物装卸和搬移报文
ICASRP	保险索赔评估与报告报文
ICSOLI	保险索赔律师指示报文
IFCSUM	运输与拼装汇总报文
IFTCCA	转运和运费计算报文
IFTDGN	危险品货物通知报文
IFTFCC	国际运输运费和其它费用报文
IFTICL	货物保险索赔报文
IFTMAN	到货通知报文
IFTMBC	订舱确认报文
IFTMBF	正式订舱报文
IFTMBP	临时订舱报文
IFTMCA	托运通知报文
IFTMCS	指示合同状态报文
IFTMIN	指示报文
IFTRIN	转运和运输费率信息报文
IFTSAI	运输计划及可获性信息报文

名称	说明
IFTSTA	国际多式联运状态报告报文
IFTSTQ	国际多式联运状态请求报文
IHCEBI	交互式健康保险资格和收益查询与应答
IHCLME	医疗保险索赔请求与应答
IMPDEF	EDI实施指南定义报文
INFCON	基础设施状态报文
INFENT	企业会计信息报文
INSDDES	发运指示报文
INSPRE	保险费报文
INSREQ	检验请求报文
INSRPT	检验报告报文
INVOIC	发票报文
INVRPT	库存报告报文
IPPOAD	保险单管理报文
IPPOMO	机动车保险单报文
ISENDS	中介系统启用或禁用报文
ITRRPT	在途报告详情报文
JAPRES	工作申请结果报文
JINFDE	工作信息要求报文
JOBAPP	工作申请建议报文
JOBCON	工作招聘确认报文
JOBMOD	工作招聘更改报文
JOBOFF	工作招聘报文
JUPREQ	确认的支付请求报文

名称	说明
LEDGER	分类账报文
LREACT	人寿再保险活动报文
LRECLM	人寿再保险索赔报文
MEDPID	人员标识报文
MEDPRE	医疗处方报文
MEDREQ	医疗服务请求报文
MEDRPT	医疗服务报告报文
MEDRUC	医疗资源的使用和费用报文
MEQPOS	运输工具和设备位置报文
MOVINS	装载指示报文
MSCONS	计量服务消费报告报文
ORDCHG	订购单变更请求报文
ORDERS	订购单报文
ORDRSP	订购单应答报文
OSTENQ	订单状态询问报文
OSTRPT	订单状态报告报文
PARTIN	参与方信息报文
PASREQ	旅游休闲产品应用状态请求
PASRSP	旅游休闲产品应用状态应答
PAXLST	乘客清单报文
PAYDUC	工资扣款通知报文
PAYEXT	扩展付款单报文
PAYMUL	多重付款通知报文
PAYORD	付款通知报文

名称	说明
PRICAT	价格/销售目录报文
PRIHIS	定价记录报文
PROCST	项目成本报告报文
PRODAT	产品数据报文
PRODEX	产品交换调节报文
PROINQ	产品询问报文
PROSRV	产品服务报文
PROTAP	项目任务计划报文
PRPAID	保险费付款报文
QUALITY	质量数据报文
QUOTES	报价报文
RDRMES	原始数据报告报文
REBORD	再保险明细报文
RECADV	收货通知报文
RECALC	再保险计算报文
RECECO	信用风险保证金报文
RECLAM	再保险索赔报文
RECORD	再保险核心数据报文
REGENT	企业登记报文
RELIST	再保险对象列表报文
REMADV	汇款通知报文
REPREM	再保险保险费报文
REQDOC	文件请求报文
REQOTE	报价请求报文

名称	说明
RESETT	再保险结算报文
RESMSG	预订报文
RESREQ	预订请求
RESRSP	预订应答
RETACC	再保险业务帐单报文
RETANN	退货通知报文
RETINS	退货说明报文
RPCALL	维修电话报文
SAFHAZ	安全与危险数据报文
SANCRT	国际货物运输政府管理报文
SKDREQ	计划请求
SKDUPD	计划应答
SLSFCT	销售预测报文
SLSRPT	销售数据报告报文
SOCAD	社会管理报文
SSIMOD	身份细目更改报文
SSRECH	工人保险记录报文
SSREGW	工人注册通知报文
STATAC	对帐单报文
STLRPT	结算交易报告报文
SUPCOT	退休金分摊通知报文
SUPMAN	退休金维护报文
SUPRES	服务商应答报文
TANSTA	液舱状态报告报文

名称	说明
TAXCON	税务管理报文
TIQREQ	旅游休闲信息查询请求
TIQRSP	旅游休闲信息查询应答
TPFREP	站场性能报文
TSDUPD	静态数据更新时间表
TUPREQ	旅游休闲数据更新请求
TUPRSP	旅游休闲数据更新应答
UTILMD	公用设施主数据报文
UTILTS	公用设施时间序列报文
VATDEC	增值税申报报文
VESDEP	船舶离港报文
WASDIS	废物处理信息报文
WKGRDC	工作许可决定报文
WKGRRE	工作许可请求报文

3.3. 结构

EDIFACT 文档在开头包括 2 个技术段，多条消息，在结尾处包括另一个技术段。以 **EDIFACT** 中的 **D96A INVOIC** 发票为例：

Interchange control Header :

UNA:+.?'

UNB+UNOC:3+Supplier_id:qualifier+SIRET number:14+Date:Hour+interchange number'

supplier_id : SIREN for French suppliers, DUN code or other id for non-French suppliers

Orange Siret = 38012986600014

Messages :

UNH Message Header 1

Document Header Section 1

...

Document Details Section 1

...

Document Footer Section 1

...

UNT (Message Trailer 1)

...

...

...

UNH (Message Header XX)

Document Header Section xx

...

Document Details Section xx

...

Document Footer Section xx

...

UNT (Message Trailer XX)

Interchange control trailer :

UNZ+XX+ interchange number' with XX = number of messages in the interchange

1. 交换层 (Interchange)

- Interchange control Header

- UNA (可选)：定义分隔符（如数据元素分隔符`+`、子元素分隔符`:`、段终止符`~`、转义字符`?`）。
- UNB (交换头)：包含语法版本、发送方、接收方、日期/时间、控制编号等。
- Interchange control trailer
 - UNZ (交换尾)：记录交换内的报文数量及控制参考号。

2. 报文层 (Message)

- UNH (消息头)：包含消息类型（如`INVOIC`）、版本号（如`D:96A`）和唯一标识。
- 数据段 (Segment)：按固定顺序排列的逻辑单元，如：
 - BGM (报文开始)：包含文档类型和编号。
 - DTM (日期/时间)：记录业务相关时间（如订单日期）。
 - NAD (名称与地址)：标识交易方（如买方、卖方）。
- UNT (消息尾)：记录报文内的段总数和控制参考号。

3. 数据段与元素的层级

- 段 (Segment)
 - 由**段标记**（3位代码，如BGM）和**数据元**组成，通过分隔符（如`+`或`:`）分隔
 - 分为**服务段**（如UNB、UNH）和**通用段**（如BGM、NAD）
- 数据元 (Data Element)
 - **简单数据元**：单一值（如订单号）
 - **复合数据元**：多个成分数据元组合（如日期+时间格式），需通过限定符明确含义（如`DTM+137:20250428:102`表示“订单日期”）
- 限定符 (Qualifier)
 - 定义数据元的业务含义（如`NAD+BY`表示“买方地址”）

4. 示例

注：如果要使用示例EDI，请务必删除 `# xxxx` 注释内容！

```
# Interchange control Header
UNA:+.?*
UNB+UNOC:3+87830609100017:14+38012986600014:14+250423:1705+17++INVOIC'
# Messages
# UNH Message Header 1
UNH+1+INVOIC:D:96A:UN:FTW200'
# 数据段 (Segment)
BGM+380+FR20250402001+9'
DTM+3:20250405:102'
PAI+::30'
FTX+AAI+++878 306 091 RCS PARIS:87830609100017?:4669A:YOFC International (France) SAS:SAS'
FTX+ABY+++300000:EUR'
FTX+PMT+++INDEMNITE FORFAITAIRE DE 40 EUROS POUR FRAIS DE RECouvreMENT'
FTX+PMD+++EN CAS DE PAIEMENT ANTICIPE PARVENANT AU VENDEUR AU MOINS 20 JOURS:AVANT LA DATE
D ECHEANCE UN ESCOMPTE EGAL A L EURIBOR EN VIGUEUR A 3:MOIS PLUS UN POINT PEUT ETRE ACCORDE
SUIVANT LE PRORATA DU NOMBRE DE:JOURS RESTANT A COURIR JUSQU A L ECHEANCE PREVUE'
FTX+AAB+++LE TAUX D INTERET DES PENALITES DE RETARD EST EGAL AU TAUX D INTERET:DE LA BANQUE
```

```

CENTRALE EUROPEENNE EN VIGUEUR AU JOUR DU REGLEMENT:MAJORE DE SEPT POINTS. LES PENALITES
DE RETARD DE PAIEMENT SONT:EXIGIBLES SANS QU UN APPEL SOIT NECESSAIRE.'
RFF+CT:IC091009'
RFF+ON:72424947'
NAD+BY+38012986600014:100++100+111 quai du President Roosevelt+Issy-les-Moulineaux++92130+FR'
RFF+VA:FR89380129866'
CTA+PD+:ZZ'
NAD+SE+87830609100017:100++YOFC INTERNATIONAL (FRANCE) SAS+37,rue des
Mathurins+Paris++75008+FR'
FII+BF+FR7630056005120512009258476+:::30056:::HSBC OF FRANCE'
RFF+VA:FR03878306091'
CTA+AP+0102030405'
TAX+VAT+VAT+ENC'
CUX+2:EUR'
LIN+1++L1092-13_36G.657A2:SA'
PIA+1+03561296608426:BP'
IMD+++:::CABLE L1092 36FO MODULO 6 FTTH'
QTY+47:2400:MTR'
MOA+125:1447.2'
PRI+CAL:0.603::NTP'
PRI+AAB:0.603::NTP'
RFF+LI:1'
RFF+DQ:FH20250405001'
DTM+171:20250425:102'
TAX+7+VAT+ENC++:::20'
UNS+S'
MOA+125:1447.2'
MOA+124:289.44'
MOA+128:1737'
MOA+9:1736.64'
MOA+ZZZ:289.44'
TAX+7+VAT++1447.2+:::20'
MOA+150:289.44'
# UNT (Message Trailer 1)
UNT+1+13514'
# Interchange control trailer
UNZ+1+17'

```

3.4. 自定义脚本

为支持非标准数据转换为EDIFACT，自定义脚本可通过编程方式将用户数据转为标准EDIFACT。以JSON为例，通常用户数据都不会满足转换数据的格式要求，如果要转换为EDIFACT，则不得不进行编程处理。

注： 示例JSON和自定义脚本篇幅较大，仅作演示，后文会讲解主要的类，细节不作解释。

JSON数据格式如下:

```
{
  "supplierId": "87830609100017",
  "siretNumber": "38012986600014",
  "headMessageReferenceNum": "13514",
  "associationCode": "FTW200",
  "documentType": "380",
  "documentIdentifier": "1812758",
  "messageFunctionCode": "9",
  "invoiceDate": "20180625",
  "paymentInstructions": "30",
  "vendorInfos":
  [
    "33-085-1143 RCS BERLIN",
    "33-085-1143",
    "CORNING CABLE SYSTEMS GMBH & CO KG",
    "GMBH"
  ],
  "capitalShares":
  [
    "504900",
    "EUR"
  ],
  "recoveryCosts":
  [
    "INDEMNITE FORFAITAIRE DE 40 EUROS POUR FRAIS DE RECouvreMENT"
  ],
  "discountConditions":
  [
    "EN CAS DE PAIEMENT ANTICIPE PARVENANT AU VENDEUR AU MOINS 20 JOURS",
    "AVANT LA DATE D ECHEANCE UN ESCOMPTE EGAL A L EURIBOR EN VIGUEUR A 3",
    "MOIS PLUS UN POINT PEUT ETRE ACCORDE SUIVANT LE PRORATA DU NOMBRE DE",
    "JOURS RESTANT A COURIR JUSQU A L ECHEANCE PREVUE"
  ],
  "penaltyConditions":
  [
    "LE TAUX D INTERET DES PENALITES DE RETARD EST EGAL AU TAUX D INTERET",
    "DE LA BANQUE CENTRALE EUROPEENNE EN VIGUEUR AU JOUR DU REGLEMENT",
    "MAJORE DE SEPT POINTS. LES PENALITES DE RETARD DE PAIEMENT SONT",
    "EXIGIBLES SANS QU UN APPEL SOIT NECESSAIRE."
  ],
  "freeTexts":
  [
```

```

    "28D-Autoliquidation"
  ],
  "contractNumber": "50026100",
  "customerOrderNumber": "AA00205076",
  "referenceDocumentNum": "109115656",
  "referenceDocumentDate": "20180425",
  "referenceDownpaymentDocumentNum": "109115656",
  "referenceDownpaymentDocumentDate": "20180425",
  "buyerCustomerId": "38012986600014",
  "buyerEnhancedParty": "100",
  "buyerCustomerName": "ORANGE:CSPCF Compta fournisseur EDI",
  "buyerStreetInfos":
  [
    "TSA 18105"
  ],
  "buyerCityName": "ROUEN CEDEX",
  "buyerPostalCode": "76721",
  "buyerCountry": "FR",
  "buyerVatRegistrationNum": "FR89380129866",
  "buyerAccountingUnitCode": "ZZ",
  "sellerVendorId": "33825323022013",
  "sellerEnhancedParty": "100",
  "sellerVendorName": "SODEXO ENTREPRISES",
  "sellerStreetInfos":
  [
    "6 RUE DE LA REDOUTE"
  ],
  "sellerCityName": "GUYANCOURT",
  "sellerPostalCode": "78280",
  "sellerCountry": "FR",
  "sellerBankAccount": "FR76XXXXXXXXXXXXXXXXXXXXXXXXX",
  "sellerInstitutionBranchNum": "BNPXXXXXXXX",
  "sellerBankName": "BNP",
  "sellerVatRegistrationNum": "FR52422778555",
  "sellerContactTel": "0102030405",
  "sellerContactName": "",
  "invoiceIssuerId": "33465323022052",
  "invoiceIssuerEnhancedParty": "100",
  "invoiceIssuerName": "SIEGE SOCIAL NGE",
  "invoiceIssuerStreetInfos":
  [
    "6 RUE DE LA TOUR"
  ],
  "invoiceIssuerCityName": "GUYANCOURT",

```

```
"invoiceIssuerPostalCode": "78280",
"invoiceIssuerCountry": "FR",
"invoiceIssuerVatRegistrationNum": "FR54822778555",
"factorId": "",
"factorEnhancedParty": "",
"factorName": "",
"factorStreetInfos":
[
  ""
],
"factorCityName": "",
"factorPostalCode": "",
"factorCountry": "FR",
"factorVatRegistrationNum": "",
"taxType": "7",
"taxRule": "ENC",
"invoiceCurrency": "EUR",
"targetCurrency": "",
"exchangeRate": "",
"exchangeRateDate": "20180425",
"paymentTermsType": "3",
"paymentTermsIdentification": "6",
"paymentTerms": "Sans deduct. jusque 01.09.2018",
"paymentReferenceDate": "5",
"paymentTermsCalculation": "1",
"paymentPeriodType": "D",
"paymentPeriodUnitNum": "60",
"paymentDate": "20180425",
"allowanceOrCharge": "",
"allowanceOrChargeIterationNum": "",
"allowanceOrChargeRate": "",
"allowanceOrChargeAmount": "",
"allowanceOrChargeUnit": "",
"allowanceOrChargePerQuantityAmount": "",
"docDetails":
[
  {
    "docLineNum": "1",
    "items":
    [
      {
        "itemNum": "0280000268",
        "itemType": "SA"
      }
    ]
  }
]
```

```
{
  "itemNum": "X12334556",
  "itemType": "BP"
},
{
  "itemNum": "7612345678900",
  "itemType": "EN"
}
],
"itemDesc": "MODULE EPISSURAGE BRASSAGE MEB72-D",
"quantity": "1",
"quantityUnit": "NPT",
"serviceDeliveryInfos":
[
  "REN2",
  "20180703",
  "20180703",
  "782977|COR|1A1EM15|21_C04_161|MCU"
],
"generalInfos":
[
  "123456"
],
"taxableAmount": "164.00",
"vatAmount": "32.80",
"unitNetPrice": "164",
"unitNetPriceType": "NTP",
"grossUnitPrice": "120",
"grossUnitPriceType": "GRP",
"customerPurchaseOrderNum": "1",
"deliveryOrder": "0088180258",
"deliveryDate": "20180703",
"taxInfos":
[
  {
    "taxType": "VAT",
    "taxRule": "ENC",
    "taxRate": "20.00"
  }
],
"allowanceOrCharge": "",
"allowanceOrChargeIterationNum": "",
"allowanceOrChargeRate": "",
"allowanceOrChargeAmount": "",
```

```

        "allowanceOrChargePerQuantityAmount": ""
    }
],
    "totalTaxableAmount": "1245.00",
    "totalTaxAmount": "1245.00",
    "totalAmount": "2490.00",
    "downpaymentTotalAmount": "",
    "downpaymentTaxableAmount": "",
    "payableAmount": "",
    "totalVatAmount": "32.80",
    "taxTypeCode": "VAT",
    "currentTaxDesc": "",
    "currentTaxAssessmentBasis": "164.00",
    "currentTaxRate": "20.00",
    "addedTaxAmount": "32.80",
    "otherTaxAmount": "32.80",
    "totalAllowanceOrCharge": "",
    "totalDiscountOrChargeAmount": "",
    "messageSegments": "1",
    "tailMessageReferenceNum": "13514"
}

```

自定义脚本如下:

```

import com.alibaba.fastjson2.JSON;
import com.alibaba.fastjson2.JSONArray;
import com.alibaba.fastjson2.JSONObject;
import org.apache.camel.component.smooks.utils.SmooksCaches;
import org.smooks.edifact.binding.d96a.ALCAAllowanceOrCharge;
import org.smooks.edifact.binding.d96a.BGMBeginningOfMessage;
import org.smooks.edifact.binding.d96a.C002DocumentMessageName;
import org.smooks.edifact.binding.d96a.C056DepartmentOrEmployeeDetails;
import org.smooks.edifact.binding.d96a.C059Street;
import org.smooks.edifact.binding.d96a.C078AccountIdentification;
import org.smooks.edifact.binding.d96a.C080PartyName;
import org.smooks.edifact.binding.d96a.C082PartyIdentificationDetails;
import org.smooks.edifact.binding.d96a.C088InstitutionIdentification;
import org.smooks.edifact.binding.d96a.C108TextLiteral;
import org.smooks.edifact.binding.d96a.C110PaymentTerms;
import org.smooks.edifact.binding.d96a.C112TermsTimeInformation;
import org.smooks.edifact.binding.d96a.C128RateDetails;
import org.smooks.edifact.binding.d96a.C186QuantityDetails;
import org.smooks.edifact.binding.d96a.C212ItemNumberIdentification;
import org.smooks.edifact.binding.d96a.C241DutyTaxFeeType;
import org.smooks.edifact.binding.d96a.C243DutyTaxFeeDetail;

```

```
import org.smooks.edifact.binding.d96a.C273ItemDescription;
import org.smooks.edifact.binding.d96a.C501PercentageDetails;
import org.smooks.edifact.binding.d96a.C504CurrencyDetails;
import org.smooks.edifact.binding.d96a.C506Reference;
import org.smooks.edifact.binding.d96a.C507DateTimePeriod;
import org.smooks.edifact.binding.d96a.C509PriceInformation;
import org.smooks.edifact.binding.d96a.C516MonetaryAmount;
import org.smooks.edifact.binding.d96a.C533DutyTaxFeeAccountDetail;
import org.smooks.edifact.binding.d96a.C534PaymentInstructionDetails;
import org.smooks.edifact.binding.d96a.C552AllowanceChargeInformation;
import org.smooks.edifact.binding.d96a.CTAContactInformation;
import org.smooks.edifact.binding.d96a.CUXCurrencies;
import org.smooks.edifact.binding.d96a.DTMDateTimePeriod;
import org.smooks.edifact.binding.d96a.E1153ReferenceQualifier;
import org.smooks.edifact.binding.d96a.E3035PartyQualifier;
import org.smooks.edifact.binding.d96a.E3139ContactFunctionCoded;
import org.smooks.edifact.binding.d96a.E4451TextSubjectQualifier;
import org.smooks.edifact.binding.d96a.E5125PriceQualifier;
import org.smooks.edifact.binding.d96a.E5153DutyTaxFeeTypeCoded;
import org.smooks.edifact.binding.d96a.E5387PriceTypeQualifier;
import org.smooks.edifact.binding.d96a.E5463AllowanceOrChargeQualifier;
import org.smooks.edifact.binding.d96a.E7143ItemNumberTypeCoded;
import org.smooks.edifact.binding.d96a.FIIFinancialInstitutionInformation;
import org.smooks.edifact.binding.d96a.FTXFreeText;
import org.smooks.edifact.binding.d96a.IMDItemDescription;
import org.smooks.edifact.binding.d96a.INVOIC;
import org.smooks.edifact.binding.d96a.Interchange;
import org.smooks.edifact.binding.d96a.LINLineItem;
import org.smooks.edifact.binding.d96a.MOAMonetaryAmount;
import org.smooks.edifact.binding.d96a.Message;
import org.smooks.edifact.binding.d96a.NADNameAndAddress;
import org.smooks.edifact.binding.d96a.PAIPaymentInstructions;
import org.smooks.edifact.binding.d96a.PATPaymentTermsBasis;
import org.smooks.edifact.binding.d96a.PCDPercentageDetails;
import org.smooks.edifact.binding.d96a.PIAAdditionalProductId;
import org.smooks.edifact.binding.d96a.PRIPriceDetails;
import org.smooks.edifact.binding.d96a.QTYQuantity;
import org.smooks.edifact.binding.d96a.RFFReference;
import org.smooks.edifact.binding.d96a.RTERateDetails;
import org.smooks.edifact.binding.d96a.TAXDutyTaxFeeDetails;
import org.smooks.edifact.binding.service.E0001SyntaxIdentifier;
import org.smooks.edifact.binding.service.E0051ControllingAgencyCoded;
import org.smooks.edifact.binding.service.E0065MessageType;
import org.smooks.edifact.binding.service.E0081SectionIdentification;
```



```
import org.smooks.edifact.binding.service.S001SyntaxIdentifier;
import org.smooks.edifact.binding.service.S002InterchangeSender;
import org.smooks.edifact.binding.service.S003InterchangeRecipient;
import org.smooks.edifact.binding.service.S004DateAndTimeOfPreparation;
import org.smooks.edifact.binding.service.S005RecipientReferencePasswordDetails;
import org.smooks.edifact.binding.service.S009MessageIdentifier;
import org.smooks.edifact.binding.service.UNA;
import org.smooks.edifact.binding.service.UNBInterchangeHeader;
import org.smooks.edifact.binding.service.UNHMessageHeader;
import org.smooks.edifact.binding.service.UNSSectionControl;
import org.smooks.edifact.binding.service.UNTMessageTrailer;
import org.smooks.edifact.binding.service.UNZInterchangeTrailer;

import javax.xml.bind.JAXBContext;
import javax.xml.bind.JAXBElement;
import javax.xml.bind.JAXBException;
import javax.xml.namespace.QName;
import java.io.ByteArrayOutputStream;
import java.math.BigDecimal;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public static boolean isEmpty(String val) {
    return val == null || val.equals("");
}

String invoiceJsonStr = exchange.getMessage().getBody(String.class);
JSONObject invoiceJson = JSON.parseObject(invoiceJsonStr);

Interchange interchange = new Interchange();

// UNA
interchange.withUNA(
    new UNA()
        .withCompositeSeparator(":")
        .withFieldSeparator("+")
        .withDecimalSeparator(".")
        .withEscapeCharacter("?")
        .withRepeatSeparator("*")
        .withSegmentTerminator("")
);

// UNB
```

```

String supplierId = invoicJson.getString("supplierId");
String siretNumber = invoicJson.getString("siretNumber");
if (!isEmpty(supplierId) && !isEmpty(siretNumber)) {
    // 获取当前日期和时间
    LocalDate date = LocalDate.now();
    LocalTime time = LocalTime.now();

    // 格式化
    String formattedDate = date.format(DateTimeFormatter.ofPattern("yyMMdd"));
    String formattedTime = time.format(DateTimeFormatter.ofPattern("HHmm"));

    interchange.withUNB(
        new UNBInterchangeHeader()
            .withS001(
                new S001SyntaxIdentifier()
                    .withE0001(E0001SyntaxIdentifier.UNOC)
                    .withE0002("3")
            )
            .withS002(
                new S002InterchangeSender()
                    // supplierId
                    .withE0004(supplierId)
                    .withE0007("14")
            )
            .withS003(
                new S003InterchangeRecipient()
                    // siretNumber
                    .withE0010(siretNumber)
                    .withE0007("14")
            )
            .withS004(
                new S004DateAndTimeOfPreparation()
                    // date
                    .withE0017(BigDecimal.valueOf(Long.valueOf(formattedDate)))
                    // time
                    .withE0019(BigDecimal.valueOf(Long.valueOf(formattedTime)))
            )
            .withE0020("17")
            .withS005(
                new S005RecipientReferencePasswordDetails()
                    .withE0022("")
            )
            .withE0026("INVOIC")
        // test e0035 = 1
    );
}

```

```

        //withE0035(new BigDecimal(1))
    );
}

Message message = new Message();

// UNH
String headMessageReferenceNum = invoicJson.getString("headMessageReferenceNum");
String associationCode = invoicJson.getString("associationCode");
if (!isEmpty(headMessageReferenceNum) && !isEmpty(associationCode)) {
    message.withContent(
        new JAXBElement<>(
            new QName("UNH"), UNHMessageHeader.class,
            new UNHMessageHeader()
                // headMessageReferenceNum
                .withE0062(headMessageReferenceNum)
                .withS009(
                    new S009MessageIdentifier()
                        .withE0065(E0065MessageType.INVOIC)
                        .withE0052("D")
                        .withE0054("96A")
                        .withE0051(E0051ControllingAgencyCoded.UN)
                        // associationCode
                        .withE0057(associationCode)
                )
            )
        );
}

INVOIC invoic = new INVOIC();

// BGM
String documentType = invoicJson.getString("documentType");
String documentIdentifier = invoicJson.getString("documentIdentifier");
String messageFunctionCode = invoicJson.getString("messageFunctionCode");
if (!isEmpty(documentType) && !isEmpty(documentIdentifier) && !isEmpty(messageFunctionCode)) {
    invoic.withBGM(
        new BGMBeginningOfMessage()
            .withC002(
                new C002DocumentMessageName()
                    // documentType
                    .withE1001(documentType)
                )
            // documentIdentifier

```

```

        .withE1004(documentIdentifier)
        // messageFunctionCode
        .withE1225(messageFunctionCode)
    );
}

// DTM
String invoiceDate = invoicJson.getString("invoiceDate");
if (!isEmpty(invoiceDate)) {
    invoic.withDTM(
        new DTMDateTimePeriod()
            .withC507(
                new C507DateTimePeriod()
                    .withE2005("3")
                    // invoiceDate
                    .withE2380(invoiceDate)
                    .withE2379("102")
            )
    );
}

// PAI
String paymentInstructions = invoicJson.getString("paymentInstructions");
if (!isEmpty(paymentInstructions)) {
    invoic.withPAI(
        new PAIPaymentInstructions()
            .withC534(
                new C534PaymentInstructionDetails()
                    // paymentInstructions
                    .withE4461(paymentInstructions)
            )
    );
}

// FTX
JSONArray vendorInfos = invoicJson.getJSONArray("vendorInfos");
if (vendorInfos != null && vendorInfos.size() > 0) {
    invoic.withFTX(
        new FTXFreeText()
            .withE4451(E4451TextSubjectQualifier.AAI)
            .withC108(
                new C108TextLiteral()
                    .withE4440(
                        // vendorInfos

```

```

        vendorInfos.toArray(new String[0])
    )
)
);
}

JSONArray capitalShares = invoicJson.getJSONArray("capitalShares");
if (capitalShares != null && capitalShares.size() > 0) {
    invoic.withFTX(
        new FTXFreeText()
            .withE4451(E4451TextSubjectQualifier.ABY)
            .withC108(
                new C108TextLiteral()
                    .withE4440(
                        // capitalShares
                        capitalShares.toArray(new String[0])
                    )
            )
    );
}

JSONArray recoveryCosts = invoicJson.getJSONArray("recoveryCosts");
if (recoveryCosts != null && recoveryCosts.size() > 0) {
    invoic.withFTX(
        new FTXFreeText()
            .withE4451(E4451TextSubjectQualifier.PMT)
            .withC108(
                new C108TextLiteral()
                    .withE4440(
                        // recoveryCosts
                        recoveryCosts.toArray(new String[0])
                    )
            )
    );
}

JSONArray discountConditions = invoicJson.getJSONArray("discountConditions");
if (discountConditions != null && discountConditions.size() > 0) {
    invoic.withFTX(
        new FTXFreeText()
            .withE4451(E4451TextSubjectQualifier.PMD)
            .withC108(
                new C108TextLiteral()
                    .withE4440(

```

```

        // discountConditions
        discountConditions.toArray(new String[0])
    )
)
);
}

JSONArray penaltyConditions = invoicJson.getJSONArray("penaltyConditions");
if (penaltyConditions != null && penaltyConditions.size() > 0) {
    invoic.withFTX(
        new FTXFreeText()
            .withE4451(E4451TextSubjectQualifier.AAB)
            .withC108(
                new C108TextLiteral()
                    .withE4440(
                        // penaltyConditions
                        penaltyConditions.toArray(new String[0])
                    )
            )
    );
}

JSONArray freeTexts = invoicJson.getJSONArray("freeTexts");
if (freeTexts != null && freeTexts.size() > 0) {
    invoic.withFTX(
        new FTXFreeText()
            .withE4451(E4451TextSubjectQualifier.ACB)
            .withC108(
                new C108TextLiteral()
                    .withE4440(
                        // freeTexts
                        freeTexts.toArray(new String[0])
                    )
            )
    );
}

INVOIC.SegGrp1 contractNumberSegGrp1 = null;

// SG1.RFF
String contractNumber = invoicJson.getString("contractNumber");
if (!isEmpty(contractNumber)) {
    if (contractNumberSegGrp1 == null) {
        contractNumberSegGrp1 = new INVOIC.SegGrp1();
    }
}

```

```

    }

    contractNumberSegGrp1.withRFF(
        new RFFReference()
            .withC506(
                new C506Reference()
                    .withE1153(E1153ReferenceQualifier.CT)
                    .withE1154(contractNumber)
            )
    );
}

if (contractNumberSegGrp1 != null) {
    invoic.withSegGrp1(contractNumberSegGrp1);
}

INVOIC.SegGrp1 customerOrderNumberSegGrp1 = null;

// SG1.RFF
String customerOrderNumber = invoicJson.getString("customerOrderNumber");
if (!isEmpty(customerOrderNumber)) {
    if (customerOrderNumberSegGrp1 == null) {
        customerOrderNumberSegGrp1 = new INVOIC.SegGrp1();
    }

    customerOrderNumberSegGrp1.withRFF(
        new RFFReference()
            .withC506(
                new C506Reference()
                    .withE1153(E1153ReferenceQualifier.ON)
                    .withE1154(customerOrderNumber)
            )
    );
}

if (customerOrderNumberSegGrp1 != null) {
    invoic.withSegGrp1(customerOrderNumberSegGrp1);
}

INVOIC.SegGrp1 referenceSegGrp1 = null;

// SG1.RFF
String referenceDocumentNum = invoicJson.getString("referenceDocumentNum");

```

```
if (!isEmpty(referenceDocumentNum)) {
    if (referenceSegGrp1 == null) {
        referenceSegGrp1 = new INVOIC.SegGrp1();
    }

    referenceSegGrp1.withRFF(
        new RFFReference()
            .withC506(
                new C506Reference()
                    .withE1153(E1153ReferenceQualifier.ACE)
                    .withE1154(referenceDocumentNum)
            )
    );
}

// SG1.DTM
String referenceDocumentDate = invoicJson.getString("referenceDocumentDate");
if (!isEmpty(referenceDocumentDate)) {
    if (referenceSegGrp1 == null) {
        referenceSegGrp1 = new INVOIC.SegGrp1();
    }

    referenceSegGrp1.withDTM(
        new DTMDateTimePeriod()
            .withC507(
                new C507DateTimePeriod()
                    .withE2005("171")
                    .withE2380(referenceDocumentDate)
                    .withE2379("102")
            )
    );
}

if (referenceSegGrp1 != null) {
    invoic.withSegGrp1(referenceSegGrp1);
}

INVOIC.SegGrp1 referenceDownpaymentSegGrp1 = null;

// SG1.RFF
String referenceDownpaymentDocumentNum = invoicJson.getString("referenceDownpaymentDocumentNum");
if (!isEmpty(referenceDownpaymentDocumentNum)) {
    if (referenceDownpaymentSegGrp1 == null) {
        referenceDownpaymentSegGrp1 = new INVOIC.SegGrp1();
    }
}
```



```

    }

    referenceDownpaymentSegGrp1.withRFF(
        new RFFReference()
            .withC506(
                new C506Reference()
                    .withE1153(E1153ReferenceQualifier.ADE)
                    .withE1154(referenceDownpaymentDocumentNum)
            )
    );
}

// SG1.DTM
String referenceDownpaymentDocumentDate = invoicJson.getString("referenceDownpaymentDocumentDate");
if (!isEmpty(referenceDownpaymentDocumentDate)) {
    if (referenceDownpaymentSegGrp1 == null) {
        referenceDownpaymentSegGrp1 = new INVOIC.SegGrp1();
    }

    referenceDownpaymentSegGrp1.withDTM(
        new DTMDatetimePeriod()
            .withC507(
                new C507datetimePeriod()
                    .withE2005("171")
                    .withE2380(referenceDownpaymentDocumentDate)
                    .withE2379("102")
            )
    );
}

if (referenceDownpaymentSegGrp1 != null) {
    invoic.withSegGrp1(referenceDownpaymentSegGrp1);
}

INVOIC.SegGrp2 buyerSegGrp2 = null;

// SG2.NAD
NADNameAndAddress buyerNAD = null;

String buyerCustomerId = invoicJson.getString("buyerCustomerId");
if (!isEmpty(buyerCustomerId)) {
    if (buyerSegGrp2 == null) {
        buyerSegGrp2 = new INVOIC.SegGrp2();
    }
}

```

```
if (buyerNAD == null) {
    buyerNAD = new NADNameAndAddress();
    buyerNAD.withE3035(E3035PartyQualifier.BY);
}

String buyerEnhancedParty = invoicJson.getString("buyerEnhancedParty");
if (!isEmpty(buyerEnhancedParty)) {
    buyerNAD.withC082(
        new C082PartyIdentificationDetails()
            .withE3039(buyerCustomerId)
            .withE1131(buyerEnhancedParty)
    );
}

String buyerCustomerName = invoicJson.getString("buyerCustomerName");
if (!isEmpty(buyerCustomerName)) {
    buyerNAD.withC080(
        new C080PartyName()
            .withE3036(buyerCustomerName)
    );
}

JSONArray buyerStreetInfos = invoicJson.getJSONArray("buyerStreetInfos");
if (buyerStreetInfos != null && buyerStreetInfos.size() > 0) {
    buyerNAD.withC059(
        new C059Street()
            .withE3042(buyerStreetInfos.toArray(new String[0]))
    );
}

String buyerCityName = invoicJson.getString("buyerCityName");
if (!isEmpty(buyerCityName)) {
    buyerNAD.withE3164(buyerCityName);
}

String buyerPostalCode = invoicJson.getString("buyerPostalCode");
if (!isEmpty(buyerPostalCode)) {
    buyerNAD.withE3251(buyerPostalCode);
}

String buyerCountry = invoicJson.getString("buyerCountry");
if (!isEmpty(buyerCountry)) {
    buyerNAD.withE3207(buyerCountry);
}
```

```

}

buyerSegGrp2.withNAD(buyerNAD);

String buyerVatRegistrationNum = invoicJson.getString("buyerVatRegistrationNum");
if (!isEmpty(buyerVatRegistrationNum)) {
    buyerSegGrp2.withSegGrp3(
        new INVOIC.SegGrp2.SegGrp3()
            .withRFF(
                new RFFReference()
                    .withC506(
                        new C506Reference()
                            .withE1153(E1153ReferenceQualifier.VA)
                            .withE1154(buyerVatRegistrationNum)
                        )
                )
    );
}

String buyerAccountingUnitCode = invoicJson.getString("buyerAccountingUnitCode");
if (!isEmpty(buyerAccountingUnitCode)) {
    buyerSegGrp2.withSegGrp5(
        new INVOIC.SegGrp2.SegGrp5()
            .withCTA(
                new CTAContactInformation()
                    .withE3139(E3139ContactFunctionCoded.PD)
                    .withC056(
                        new C056DepartmentOrEmployeeDetails()
                            .withE3412(buyerAccountingUnitCode)
                        )
            )
    );
}

if (buyerSegGrp2 != null) {
    invoic.withSegGrp2(buyerSegGrp2);
}

// SG2.NAD FII
INVOIC.SegGrp2 sellerSegGrp2 = null;
NADNameAndAddress sellerNAD = null;

String sellerVendorId = invoicJson.getString("sellerVendorId");

```

```
if (!isEmpty(sellerVendorId)) {
    if (sellerSegGrp2 == null) {
        sellerSegGrp2 = new INVOIC.SegGrp2();
    }

    if (sellerNAD == null) {
        sellerNAD = new NADNameAndAddress();
        sellerNAD.withE3035(E3035PartyQualifier.SE);
    }

    String sellerEnhancedParty = invoicJson.getString("sellerEnhancedParty");
    if (!isEmpty(sellerEnhancedParty)) {
        sellerNAD.withC082(
            new C082PartyIdentificationDetails()
                .withE3039(sellerVendorId)
                .withE1131(sellerEnhancedParty)
        );
    }

    String sellerVendorName = invoicJson.getString("sellerVendorName");
    if (!isEmpty(sellerVendorName)) {
        sellerNAD.withC080(
            new C080PartyName()
                .withE3036(sellerVendorName)
        );
    }

    JSONArray sellerStreetInfos = invoicJson.getJSONArray("sellerStreetInfos");
    if (sellerStreetInfos != null && sellerStreetInfos.size() > 0) {
        sellerNAD.withC059(
            new C059Street()
                .withE3042(sellerStreetInfos.toArray(new String[0]))
        );
    }

    String sellerCityName = invoicJson.getString("sellerCityName");
    if (!isEmpty(sellerCityName)) {
        sellerNAD.withE3164(sellerCityName);
    }

    String sellerPostalCode = invoicJson.getString("sellerPostalCode");
    if (!isEmpty(sellerPostalCode)) {
        sellerNAD.withE3251(sellerPostalCode);
    }
}
```

```
String sellerCountry = invoicJson.getString("sellerCountry");
if (!isEmpty(sellerCountry)) {
    sellerNAD.withE3207(sellerCountry);
}

sellerSegGrp2.withNAD(sellerNAD);

FIIFinancialInstitutionInformation sellerFFI = null;

String sellerBankAccount = invoicJson.getString("sellerBankAccount");
if (!isEmpty(sellerBankAccount)) {
    if (sellerFFI == null) {
        sellerFFI = new FIIFinancialInstitutionInformation()
            .withE3035(E3035PartyQualifier.BF);
    }

    sellerFFI.withC078(
        new C078AccountIdentification()
            .withE3194(sellerBankAccount)
    );
}

String sellerBankName = invoicJson.getString("sellerBankName");
String sellerInstitutionBranchNum = invoicJson.getString("sellerInstitutionBranchNum");
if (!isEmpty(sellerBankName) && !isEmpty(sellerInstitutionBranchNum)) {
    if (sellerFFI == null) {
        sellerFFI = new FIIFinancialInstitutionInformation();
    }

    sellerFFI.withC088(
        new C088InstitutionIdentification()
            .withE3434(sellerInstitutionBranchNum)
            .withE3432(sellerBankName)
    );
}

if (sellerFFI != null) {
    sellerSegGrp2.withFII(sellerFFI);
}

String sellerVatRegistrationNum = invoicJson.getString("sellerVatRegistrationNum");
if (!isEmpty(sellerVatRegistrationNum)) {
    sellerSegGrp2.withSegGrp3(
```

```
        new INVOIC.SegGrp2.SegGrp3()
            .withRFF(
                new RFFReference()
                    .withC506(
                        new C506Reference()
                            .withE1153(E1153ReferenceQualifier.VA)
                            .withE1154(sellerVatRegistrationNum)
                        )
                    )
            );
    }

    CTAContactInformation sellerCTA = null;
    C056DepartmentOrEmployeeDetails sellerC056 = null;

    String sellerContactName = invoicJson.getString("sellerContactName");
    if (!isEmpty(sellerContactName)) {
        if (sellerCTA == null) {
            sellerCTA = new CTAContactInformation();
            sellerCTA.withE3139(E3139ContactFunctionCoded.AP);
        }

        if (sellerC056 == null) {
            sellerC056 = new C056DepartmentOrEmployeeDetails();
        }

        sellerC056.withE3412(sellerContactName);
        sellerCTA.withC056(sellerC056);
    }

    String sellerContactTel = invoicJson.getString("sellerContactTel");
    if (!isEmpty(sellerContactTel)) {
        if (sellerCTA == null) {
            sellerCTA = new CTAContactInformation();
            sellerCTA.withE3139(E3139ContactFunctionCoded.AP);
        }

        if (sellerC056 == null) {
            sellerC056 = new C056DepartmentOrEmployeeDetails();
        }

        sellerC056.withE3413(sellerContactTel);
        sellerCTA.withC056(sellerC056);
    }
}
```

```
if (sellerCTA != null) {
    sellerSegGrp2.withSegGrp5(
        new INVOIC.SegGrp2.SegGrp5().withCTA(sellerCTA)
    );
}
}

if (sellerSegGrp2 != null) {
    invoic.withSegGrp2(sellerSegGrp2);
}

// SG2.NAD
INVOIC.SegGrp2 invoiceIssuerSegGrp2 = null;
NADNameAndAddress invoiceIssuerNAD = null;

String invoiceIssuerId = invoicJson.getString("invoiceIssuerId");
if (!isEmpty(invoiceIssuerId)) {
    if (invoiceIssuerSegGrp2 == null) {
        invoiceIssuerSegGrp2 = new INVOIC.SegGrp2();
    }

    if (invoiceIssuerNAD == null) {
        invoiceIssuerNAD = new NADNameAndAddress();
        invoiceIssuerNAD.withE3035(E3035PartyQualifier.II);
    }

    String invoiceIssuerEnhancedParty = invoicJson.getString("invoiceIssuerEnhancedParty");
    if (!isEmpty(invoiceIssuerEnhancedParty)) {
        invoiceIssuerNAD.withC082(
            new C082PartyIdentificationDetails()
                .withE3039(invoiceIssuerId)
                .withE1131(invoiceIssuerEnhancedParty)
        );
    }

    String invoiceIssuerName = invoicJson.getString("invoiceIssuerName");
    if (!isEmpty(invoiceIssuerName)) {
        invoiceIssuerNAD.withC080(
            new C080PartyName()
                .withE3036(invoiceIssuerName)
        );
    }
}
```

```

JSONArray invoiceIssuerStreetInfos = invoicJson.getJSONArray("invoiceIssuerStreetInfos");
if (invoiceIssuerStreetInfos != null && invoiceIssuerStreetInfos.size() > 0) {
    invoiceIssuerNAD.withC059(
        new C059Street()
            .withE3042(invoiceIssuerStreetInfos.toArray(new String[0]))
    );
}

String invoiceIssuerCityName = invoicJson.getString("invoiceIssuerCityName");
if (!isEmpty(invoiceIssuerCityName)) {
    invoiceIssuerNAD.withE3164(invoiceIssuerCityName);
}

String invoiceIssuerPostalCode = invoicJson.getString("invoiceIssuerPostalCode");
if (!isEmpty(invoiceIssuerPostalCode)) {
    invoiceIssuerNAD.withE3251(invoiceIssuerPostalCode);
}

String invoiceIssuerCountry = invoicJson.getString("invoiceIssuerCountry");
if (!isEmpty(invoiceIssuerCountry)) {
    invoiceIssuerNAD.withE3207(invoiceIssuerCountry);
}

if (invoiceIssuerNAD != null) {
    invoiceIssuerSegGrp2.withNAD(invoiceIssuerNAD);
}

String invoiceIssuerVatRegistrationNum = invoicJson.getString("invoiceIssuerVatRegistrationNum");
if (!isEmpty(invoiceIssuerVatRegistrationNum)) {
    invoiceIssuerSegGrp2.withSegGrp3(
        new INVOIC.SegGrp2.SegGrp3()
            .withRFF(
                new RFFReference()
                    .withC506(
                        new C506Reference()
                            .withE1153(E1153ReferenceQualifier.VA)
                            .withE1154(invoiceIssuerVatRegistrationNum)
                        )
            )
    );
}

if (invoiceIssuerSegGrp2 != null) {

```



```
    invoic.withSegGrp2(invoiceIssuerSegGrp2);
}

// SG2.NAD
INVOIC.SegGrp2 factorSegGrp2 = null;
NADNameAndAddress factorNAD = null;

String factorId = invoicJson.getString("factorId");
if (!isEmpty(factorId)) {
    if (factorSegGrp2 == null) {
        factorSegGrp2 = new INVOIC.SegGrp2();
    }

    if (factorNAD == null) {
        factorNAD = new NADNameAndAddress();
        factorNAD.withE3035(E3035PartyQualifier.RB);
    }

    String factorEnhancedParty = invoicJson.getString("factorEnhancedParty");
    if (!isEmpty(factorEnhancedParty)) {
        factorNAD.withC082(
            new C082PartyIdentificationDetails()
                .withE3039(factorId)
                .withE1131(factorEnhancedParty)
        );
    }

    String factorName = invoicJson.getString("factorName");
    if (!isEmpty(factorName)) {
        factorNAD.withC080(
            new C080PartyName()
                .withE3036(factorName)
        );
    }

    JSONArray factorStreetInfos = invoicJson.getJSONArray("factorStreetInfos");
    if (factorStreetInfos != null && factorStreetInfos.size() > 0) {
        factorNAD.withC059(
            new C059Street()
                .withE3042(factorStreetInfos.toArray(new String[0]))
        );
    }

    String factorCityName = invoicJson.getString("factorCityName");
```

```

if (!isEmpty(factorCityName)) {
    factorNAD.withE3164(factorCityName);
}

String factorPostalCode = invoicJson.getString("factorPostalCode");
if (!isEmpty(factorPostalCode)) {
    factorNAD.withE3251(factorPostalCode);
}

String factorCountry = invoicJson.getString("factorCountry");
if (!isEmpty(factorCountry)) {
    factorNAD.withE3207(factorCountry);
}

factorSegGrp2.withNAD(factorNAD);

String factorVatRegistrationNum = invoicJson.getString("factorVatRegistrationNum");
if (!isEmpty(factorVatRegistrationNum)) {
    factorSegGrp2.withSegGrp3(
        new INVOIC.SegGrp2.SegGrp3()
            .withRFF(
                new RFFReference()
                    .withC506(
                        new C506Reference()
                            .withE1153(E1153ReferenceQualifier.VA)
                            .withE1154(factorVatRegistrationNum)
                        )
                    )
            )
    );
}
}

if (factorSegGrp2 != null) {
    invoic.withSegGrp2(factorSegGrp2);
}

// SG6.TAX
INVOIC.SegGrp6 taxSegGrp6 = null;
TAXDutyTaxFeeDetails taxDetails = null;

String taxType = invoicJson.getString("taxType");
if (!isEmpty(taxType)) {
    if (taxSegGrp6 == null) {
        taxSegGrp6 = new INVOIC.SegGrp6();
    }
}

```

```
}

if (taxDetails == null) {
    taxDetails = new TAXDutyTaxFeeDetails().withE5283(taxType);
}

taxDetails.withC241(
    new C241DutyTaxFeeType()
//                                .withE5152(taxType)
    .withE5153(E5153DutyTaxFeeTypeCoded.VAT)
);

String taxRule = invoicJson.getString("taxRule");
if (!isEmpty(taxRule)) {
    taxDetails.withC533(
        new C533DutyTaxFeeAccountDetail()
            .withE5289(taxRule)
    );
}

taxSegGrp6.withTAX(taxDetails);
}

if (taxSegGrp6 != null) {
    invoic.withSegGrp6(taxSegGrp6);
}

//SG7.CUX
INVOIC.SegGrp7 invoicSegGrp7 = null;

String invoiceCurrency = invoicJson.getString("invoiceCurrency");
if (!isEmpty(invoiceCurrency)) {
    if (invoicSegGrp7 == null) {
        invoicSegGrp7 = new INVOIC.SegGrp7();
    }

    invoicSegGrp7.withCUX(
        new CUXCurrencies()
            .withC504(
                new C504CurrencyDetails()
                    .withE6347("2")
                    .withE6345(invoiceCurrency)
            )
    );
}
```

```
}

String targetCurrency = invoicJson.getString("targetCurrency");
if (!isEmpty(targetCurrency)) {
    if (invoicSegGrp7 == null) {
        invoicSegGrp7 = new INVOIC.SegGrp7();
    }

    invoicSegGrp7.withCUX(
        new CUXCurrencies()
            .withC504(
                new C504CurrencyDetails()
                    .withE6347("3")
                    .withE6345(targetCurrency)
            )
    );
}

BigDecimal exchangeRate = invoicJson.getBigDecimal("exchangeRate");
if (exchangeRate != null) {
    if (invoicSegGrp7 == null) {
        invoicSegGrp7 = new INVOIC.SegGrp7();
    }

    invoicSegGrp7.withCUX(
        new CUXCurrencies()
            .withE5402(exchangeRate)
    );
}

String exchangeRateDate = invoicJson.getString("exchangeRateDate");
if (!isEmpty(exchangeRateDate)) {
    if (invoicSegGrp7 == null) {
        invoicSegGrp7 = new INVOIC.SegGrp7();
    }

    invoicSegGrp7.withDTM(
        new DTMDateTimePeriod()
            .withC507(
                new C507DateTimePeriod()
                    .withE2005("134")
                    .withE2380(exchangeRateDate)
                    .withE2379("102")
            )
    );
}
```

```
);  
}  
  
if (invoicSegGrp7 != null) {  
    invoic.withSegGrp7(invoicSegGrp7);  
}  
  
// SG8.PAT DTM  
INVOIC.SegGrp8 paymentSegGrp8 = null;  
PATPaymentTermsBasis paymentPAT = null;  
  
String paymentTermsType = invoicJson.getString("paymentTermsType");  
if (!isEmpty(paymentTermsType)) {  
    if (paymentSegGrp8 == null) {  
        paymentSegGrp8 = new INVOIC.SegGrp8();  
    }  
  
    if (paymentPAT == null) {  
        paymentPAT = new PATPaymentTermsBasis().withE4279(paymentTermsType);  
    }  
  
    C110PaymentTerms paymentC110 = null;  
  
    String paymentTermsIdentification = invoicJson.getString("paymentTermsIdentification");  
    if (!isEmpty(paymentTermsIdentification)) {  
        if (paymentC110 == null) {  
            paymentC110 = new C110PaymentTerms();  
        }  
  
        paymentC110.withE4277(paymentTermsIdentification);  
    }  
  
    String paymentTerms = invoicJson.getString("paymentTerms");  
    if (!isEmpty(paymentTerms)) {  
        if (paymentC110 == null) {  
            paymentC110 = new C110PaymentTerms();  
        }  
  
        paymentC110.withE4276(paymentTerms);  
    }  
  
    if (paymentC110 != null) {  
        paymentPAT.withC110(paymentC110);  
    }  
}
```

```
C112TermsTimelnformation paymentC112 = null;

String paymentReferenceDate = invoiceJson.getString("paymentReferenceDate");
if (!isEmpty(paymentReferenceDate)) {
    if (paymentC112 == null) {
        paymentC112 = new C112TermsTimelnformation();
    }

    paymentC112.withE2475(paymentReferenceDate);
}

String paymentTermsCalculation = invoiceJson.getString("paymentTermsCalculation");
if (!isEmpty(paymentTermsCalculation)) {
    if (paymentC112 == null) {
        paymentC112 = new C112TermsTimelnformation();
    }

    paymentC112.withE2009(paymentTermsCalculation);
}

String paymentPeriodType = invoiceJson.getString("paymentPeriodType");
if (!isEmpty(paymentPeriodType)) {
    if (paymentC112 == null) {
        paymentC112 = new C112TermsTimelnformation();
    }

    paymentC112.withE2151(paymentPeriodType);
}

BigDecimal paymentPeriodUnitNum = invoiceJson.getBigDecimal("paymentPeriodUnitNum");
if (paymentPeriodUnitNum != null) {
    if (paymentC112 == null) {
        paymentC112 = new C112TermsTimelnformation();
    }

    paymentC112.withE2152(paymentPeriodUnitNum);
}

if (paymentC112 != null) {
    paymentPAT.withC112(paymentC112);
}

if (paymentPAT != null) {
```

```

        paymentSegGrp8.withPAT(paymentPAT);
    }

    String paymentDate = invoicJson.getString("paymentDate");
    if (!isEmpty(paymentDate)) {
        paymentSegGrp8.withDTM(
            new DTMDateTimePeriod()
                .withC507(
                    new C507DateTimePeriod()
                        .withE2005("138")
                        .withE2380(paymentDate)
                        .withE2379("102")
                )
        );
    }
}

if (paymentSegGrp8 != null) {
    invoic.withSegGrp8(paymentSegGrp8);
}

// SG15.ALC SG18.PCD SG19.MOA SG20.RTE
INVOIC.SegGrp15 alcSegGrp15 = null;

String allowanceOrCharge = invoicJson.getString("allowanceOrCharge");
if (!isEmpty(allowanceOrCharge)) {
    if (alcSegGrp15 == null) {
        alcSegGrp15 = new INVOIC.SegGrp15();
    }

    ALCAAllowanceOrCharge alcAllowanceOrCharge = new ALCAAllowanceOrCharge()
        .withE5463(E5463AllowanceOrChargeQualifier.fromValue(allowanceOrCharge))
        .withC552(
            new C552AllowanceChargeInformation()
                .withE1230(allowanceOrCharge)
        );

    String allowanceOrChargeIterationNum = invoicJson.getString("allowanceOrChargeIterationNum");
    if (!isEmpty(allowanceOrChargeIterationNum)) {
        alcAllowanceOrCharge.withE1227(allowanceOrChargeIterationNum);
    }

    alcSegGrp15.withALC(alcAllowanceOrCharge);
}

```

```

BigDecimal allowanceOrChargeRate = invoicJson.getBigDecimal("allowanceOrChargeRate");
if (allowanceOrChargeRate != null) {
    alcSegGrp15.withSegGrp18(
        new INVOIC.SegGrp15.SegGrp18()
            .withPCD(
                new PCDPercentageDetails()
                    .withC501(
                        new C501PercentageDetails()
                            .withE5482(allowanceOrChargeRate)
                        )
                    )
            )
    );
}

INVOIC.SegGrp15.SegGrp19 alcSegGrp19 = null;

BigDecimal allowanceOrChargeAmount = invoicJson.getBigDecimal("allowanceOrChargeAmount");
if (allowanceOrChargeAmount != null) {
    if (alcSegGrp19 == null) {
        alcSegGrp19 = new INVOIC.SegGrp15.SegGrp19();
    }

    alcSegGrp19.withMOA(
        new MOAMonetaryAmount()
            .withC516(
                new C516MonetaryAmount()
                    .withE5004(allowanceOrChargeAmount)
                )
            )
    );
}

BigDecimal allowanceOrChargeUnit = invoicJson.getBigDecimal("allowanceOrChargeUnit");
if (allowanceOrChargeUnit != null) {
    if (alcSegGrp19 == null) {
        alcSegGrp19 = new INVOIC.SegGrp15.SegGrp19();
    }

    alcSegGrp19.withMOA(
        new MOAMonetaryAmount()
            .withC516(
                new C516MonetaryAmount()
                    .withE5004(allowanceOrChargeUnit)
                )
            )
    );
}

```



```

    }

    if (alcSegGrp19 != null) {
        alcSegGrp15.withSegGrp19(alcSegGrp19);
    }

    BigDecimal allowanceOrChargePerQuantityAmount =
invoicJson.getBigDecimal("allowanceOrChargePerQuantityAmount");
    if (allowanceOrChargePerQuantityAmount != null) {
        alcSegGrp15.withSegGrp20(
            new INVOIC.SegGrp15.SegGrp20()
                .withRTE(
                    new RTERateDetails()
                        .withC128(
                            new C128RateDetails()
                                .withE5420(allowanceOrChargePerQuantityAmount)
                        )
                )
        );
    }
}

if (alcSegGrp15 != null) {
    invoic.withSegGrp15(alcSegGrp15);
}

JSONArray docDetails = invoicJson.getJSONArray("docDetails");
for (int i = 0; i < docDetails.size(); i++) {
    JSONObject doc = docDetails.getJSONObject(i);

    if (doc != null) {
        INVOIC.SegGrp25 docSegGrp25 = null;

        // SG25.LIN SG25.PIA SG25.IMD SG25.QTY SG25.FTX SG26.MOA SG28.PRI SG29.RFF SG33.TAX
        BigDecimal docLineNum = doc.getBigDecimal("docLineNum");
        if (docLineNum != null) {
            if (docSegGrp25 == null) {
                docSegGrp25 = new INVOIC.SegGrp25();
            }

            JSONArray items = doc.getJSONArray("items");
            if (items != null && items.size() > 0) {
                PIAAdditionalProductId[] piaAdditionalProductIds = new PIAAdditionalProductId[items.size()];
                for (int j = 0; j < items.size(); j++) {

```

```

JSONObject item = items.getJSONObject(j);

if (item.containsKey("itemNum") && item.containsKey("itemType")) {
    String itemNum = item.getString("itemNum");
    String itemType = item.getString("itemType");

    if (j == 0) {
        docSegGrp25.withLIN(
            new LINLineItem()
                .withE1082(docLineNum)
                .withC212(
                    new C212ItemNumberIdentification()
                        .withE7140(itemNum)
                        .withE7143(E7143ItemNumberTypeCoded.fromValue(itemType))
                )
        );
    } else {
        PIAAdditionalProductId piaAdditionalProductId = new PIAAdditionalProductId()
            .withE4347("1")
            .withC212(
                new C212ItemNumberIdentification()
                    .withE7140(itemNum)
                    .withE7143(E7143ItemNumberTypeCoded.fromValue(itemType))
            );
        piaAdditionalProductIds[j] = piaAdditionalProductId;
    }
}

docSegGrp25.withPIA(
    piaAdditionalProductIds
);

String itemDesc = doc.getString("itemDesc");
if (!isEmpty(itemDesc)) {
    docSegGrp25.withIMD(
        new IMDItemDescription()
            .withC273(
                new C273ItemDescription()
                    .withE7008(itemDesc)
            )
    );
}

```

```

BigDecimal quantity = doc.getBigDecimal("quantity");
String quantityUnit = doc.getString("quantityUnit");
if (quantity != null && !isEmpty(quantityUnit)) {
    docSegGrp25.withQTY(
        new QTYQuantity()
            .withC186(
                new C186QuantityDetails()
                    .withE6063("47")
                    .withE6060(quantity)
                    .withE6411(quantityUnit)
            )
    );
}

JSONArray serviceDeliveryInfos = doc.getJSONArray("serviceDeliveryInfos");
if (serviceDeliveryInfos != null && serviceDeliveryInfos.size() > 0) {
    docSegGrp25.withFTX(
        new FTXFreeText()
            .withE4451(E4451TextSubjectQualifier.ACF)
            .withC108(
                new C108TextLiteral()
                    .withE4440(serviceDeliveryInfos.toArray(new String[0]))
            )
    );
}

JSONArray generalInfos = doc.getJSONArray("generalInfos");
if (generalInfos != null && generalInfos.size() > 0) {
    docSegGrp25.withFTX(
        new FTXFreeText()
            .withE4451(E4451TextSubjectQualifier.AAI)
            .withC108(
                new C108TextLiteral()
                    .withE4440(generalInfos.toArray(new String[0]))
            )
    );
}

BigDecimal taxableAmount = doc.getBigDecimal("taxableAmount");
if (taxableAmount != null) {
    docSegGrp25.withSegGrp26(
        new INVOIC.SegGrp25.SegGrp26()
            .withMOA(

```

```

        new MOAMonetaryAmount()
            .withC516(
                new C516MonetaryAmount()
                    .withE5025("125")
                    .withE5004(taxableAmount)
            )
    );
}

BigDecimal vatAmount = doc.getBigDecimal("vatAmount");
if (vatAmount != null) {
    docSegGrp25.withSegGrp26(
        new INVOIC.SegGrp25.SegGrp26()
            .withMOA(
                new MOAMonetaryAmount()
                    .withC516(
                        new C516MonetaryAmount()
                            .withE5025("150")
                            .withE5004(vatAmount)
                    )
            )
    );
}

BigDecimal unitNetPrice = doc.getBigDecimal("unitNetPrice");
String unitNetPriceType = doc.getString("unitNetPriceType");
if (unitNetPrice != null && !isEmpty(unitNetPriceType)) {
    docSegGrp25.withSegGrp28(
        new INVOIC.SegGrp25.SegGrp28()
            .withPRI(
                new PRIPriceDetails()
                    .withC509(
                        new C509PriceInformation()
                            .withE5125(E5125PriceQualifier.CAL)
                            .withE5118(unitNetPrice)
                            .withE5387(E5387PriceTypeQualifier.valueOf(unitNetPriceType))
                    )
            )
    );
}

BigDecimal grossUnitPrice = doc.getBigDecimal("grossUnitPrice");
String grossUnitPriceType = doc.getString("unitNetPriceType");

```

```

if (grossUnitPrice != null && !isEmpty(grossUnitPriceType)) {
    docSegGrp25.withSegGrp28(
        new INVOIC.SegGrp25.SegGrp28()
            .withPRI(
                new PRIPriceDetails()
                    .withC509(
                        new C509PriceInformation()
                            .withE5125(E5125PriceQualifier.AAB)
                            .withE5118(grossUnitPrice)
                            .withE5387(E5387PriceTypeQualifier.valueOf(grossUnitPriceType))
                        )
                    )
            )
    );
}

String customerPurchaseOrderNum = doc.getString("customerPurchaseOrderNum");
if (!isEmpty(customerPurchaseOrderNum)) {
    docSegGrp25.withSegGrp29(
        new INVOIC.SegGrp25.SegGrp29()
            .withRFF(
                new RFFReference()
                    .withC506(
                        new C506Reference()
                            .withE1153(E1153ReferenceQualifier.LI)
                            .withE1154(customerPurchaseOrderNum)
                        )
                    )
            )
    );
}

String deliveryOrder = doc.getString("deliveryOrder");
String deliveryDate = doc.getString("deliveryDate");
if (!isEmpty(deliveryOrder) && !isEmpty(deliveryDate)) {
    docSegGrp25.withSegGrp29(
        new INVOIC.SegGrp25.SegGrp29()
            .withRFF(
                new RFFReference()
                    .withC506(
                        new C506Reference()
                            .withE1153(E1153ReferenceQualifier.DQ)
                            .withE1154(deliveryOrder)
                        )
                    )
            )
        .withDTM(

```

```

        new DTMDDateTimePeriod()
            .withC507(
                new C507DateTimePeriod()
                    .withE2005("171")
                    .withE2380(deliveryDate)
                    .withE2379("102")
            )
        );
    }

JSONArray taxInfos = doc.getJSONArray("taxInfos");
if (taxInfos != null && taxInfos.size() > 0) {
    TAXDutyTaxFeeDetails[] taxDutyTaxFeeDetails = new TAXDutyTaxFeeDetails[taxInfos.size()];
    for (int k = 0; k < taxInfos.size(); k++) {
        JSONObject taxInfo = taxInfos.getJSONObject(k);

        String _taxType = taxInfo.getString("taxType");
        String _taxRule = taxInfo.getString("taxRule");
        String _taxRate = taxInfo.getString("taxRate");

        taxDutyTaxFeeDetails[k] = new TAXDutyTaxFeeDetails()
            .withE5283("7")
            .withC241(
                new C241DutyTaxFeeType()
                    .withE5153(E5153DutyTaxFeeTypeCoded.valueOf(_taxType))
            )
            .withC533(
                new C533DutyTaxFeeAccountDetail()
                    .withE5289(_taxRule)
            )
            .withC243(
                new C243DutyTaxFeeDetail()
                    .withE5278(_taxRate)
            );
    }

    if (taxDutyTaxFeeDetails.length == 1) {
        docSegGrp25.withSegGrp33(
            new INVOIC.SegGrp25.SegGrp33()
                .withTAX(
                    taxDutyTaxFeeDetails[0]
                )
        );
    }
}

```

```

    } else if (taxDutyTaxFeeDetails.length > 1) {
        docSegGrp25.withSegGrp33(
            new INVOIC.SegGrp25.SegGrp33()
                .withTAX(taxDutyTaxFeeDetails[0])
                .withTAX(taxDutyTaxFeeDetails[1])
        );
    }
}

if (docSegGrp25 != null) {
    invoic.withSegGrp25(docSegGrp25);
}

// SG40.PCD SG41.MOA SG42.RTE
INVOIC.SegGrp25 docAlcSegGrp25 = null;

String _allowanceOrCharge = doc.getString("allowanceOrCharge");
if (!isEmpty(_allowanceOrCharge)) {
    if (docAlcSegGrp25 == null) {
        docAlcSegGrp25 = new INVOIC.SegGrp25();
    }

    INVOIC.SegGrp25.SegGrp38 segGrp38 = new INVOIC.SegGrp25.SegGrp38();

    String _allowanceOrChargeIterationNum = doc.getString("allowanceOrChargeIterationNum");
    if (!isEmpty(_allowanceOrChargeIterationNum)) {
        segGrp38.withALC(
            new ALCAAllowanceOrCharge()
                .withE5463(E5463AllowanceOrChargeQualifier.fromValue(_allowanceOrCharge))
                .withC552(
                    new C552AllowanceChargeInformation()
                        .withE1230(_allowanceOrCharge)
                )
                .withE1227(_allowanceOrChargeIterationNum)
        );
    }

    BigDecimal _allowanceOrChargeRate = doc.getBigDecimal("allowanceOrChargeRate");
    if (_allowanceOrChargeRate != null) {
        segGrp38.withSegGrp40(
            new INVOIC.SegGrp25.SegGrp38.SegGrp40()
                .withPCD(
                    new PCDPercentageDetails()

```

```

        .withC501(
            new C501PercentageDetails()
                .withE5482(_allowanceOrChargeRate)
        )
    );
}

BigDecimal _allowanceOrChargeAmount = doc.getBigDecimal("allowanceOrChargeAmount");
if (_allowanceOrChargeAmount != null) {
    segGrp38.withSegGrp41(
        new INVOIC.SegGrp25.SegGrp38.SegGrp41()
            .withMOA(
                new MOAMonetaryAmount()
                    .withC516(
                        new C516MonetaryAmount()
                            .withE5025("8")
                            .withE5004(_allowanceOrChargeAmount)
                    )
            )
    );
}

BigDecimal _allowanceOrChargePerQuantityAmount =
doc.getBigDecimal("allowanceOrChargePerQuantityAmount");
if (_allowanceOrChargePerQuantityAmount != null) {
    segGrp38.withSegGrp42(
        new INVOIC.SegGrp25.SegGrp38.SegGrp42()
            .withRTE(
                new RTERateDetails()
                    .withC128(
                        new C128RateDetails()
                            .withE5420(_allowanceOrChargePerQuantityAmount)
                    )
            )
    );
}

docAlcSegGrp25.withSegGrp38(segGrp38);
}

if (docAlcSegGrp25 != null) {
    invoic.withSegGrp25(docAlcSegGrp25);
}

```



```
    }  
  }  
  
  invoic.withUNS(  
    new UNSSectionControl()  
      .withE0081(E0081SectionIdentification.S)  
  );  
  
  // SG48.MOA  
  INVOIC.SegGrp48 totalTaxableAmountSegGrp48 = null;  
  
  BigDecimal totalTaxableAmount = invoicJson.getBigDecimal("totalTaxableAmount");  
  
  // TODO  
  if (totalTaxableAmount != null) {  
    if (totalTaxableAmountSegGrp48 == null) {  
      totalTaxableAmountSegGrp48 = new INVOIC.SegGrp48();  
    }  
  
    totalTaxableAmountSegGrp48.withMOA(  
      new MOAMonetaryAmount()  
        .withC516(  
          new C516MonetaryAmount()  
            .withE5025("125")  
            .withE5004(totalTaxableAmount)  
          )  
    );  
  }  
  
  if (totalTaxableAmountSegGrp48 != null) {  
    invoic.withSegGrp48(totalTaxableAmountSegGrp48);  
  }  
  
  INVOIC.SegGrp48 totalTaxAmountSegGrp48 = null;  
  
  // TODO  
  BigDecimal totalTaxAmount = invoicJson.getBigDecimal("totalTaxAmount");  
  if (totalTaxAmount != null) {  
    if (totalTaxAmountSegGrp48 == null) {  
      totalTaxAmountSegGrp48 = new INVOIC.SegGrp48();  
    }  
  
    totalTaxAmountSegGrp48.withMOA(  
      new MOAMonetaryAmount()
```

```
        .withC516(  
            new C516MonetaryAmount()  
                .withE5025("124")  
                .withE5004(totalTaxAmount)  
        )  
    );  
}  
  
if (totalTaxAmountSegGrp48 != null) {  
    invoic.withSegGrp48(totalTaxAmountSegGrp48);  
}  
  
INVOIC.SegGrp48 totalAmountSegGrp48 = null;  
  
// TODO  
BigDecimal totalAmount = invoicJson.getBigDecimal("totalAmount");  
if (totalAmount != null) {  
    if (totalAmountSegGrp48 == null) {  
        totalAmountSegGrp48 = new INVOIC.SegGrp48();  
    }  
  
    totalAmountSegGrp48.withMOA(  
        new MOAMonetaryAmount()  
            .withC516(  
                new C516MonetaryAmount()  
                    .withE5025("128")  
                    .withE5004(totalAmount)  
            )  
    );  
}  
  
if (totalAmountSegGrp48 != null) {  
    invoic.withSegGrp48(totalAmountSegGrp48);  
}  
  
INVOIC.SegGrp48 downpaymentTotalAmountSegGrp48 = null;  
  
BigDecimal downpaymentTotalAmount = invoicJson.getBigDecimal("downpaymentTotalAmount");  
if (downpaymentTotalAmount != null) {  
    if (downpaymentTotalAmountSegGrp48 == null) {  
        downpaymentTotalAmountSegGrp48 = new INVOIC.SegGrp48();  
    }  
  
    downpaymentTotalAmountSegGrp48.withMOA(  
        new MOAMonetaryAmount()  
            .withC516(  
                new C516MonetaryAmount()  
                    .withE5025("128")  
                    .withE5004(downpaymentTotalAmount)  
            )  
    );  
}
```

```
        new MOAMonetaryAmount()
            .withC516(
                new C516MonetaryAmount()
                    .withE5025("113")
                    .withE5004(downpaymentTotalAmount)
            )
    );
}

if (downpaymentTotalAmountSegGrp48 != null) {
    invoic.withSegGrp48(downpaymentTotalAmountSegGrp48);
}

INVOIC.SegGrp48 downpaymentTaxableAmountSegGrp48 = null;

BigDecimal downpaymentTaxableAmount = invoicJson.getBigDecimal("downpaymentTaxableAmount");
if (downpaymentTaxableAmount != null) {
    if (downpaymentTaxableAmountSegGrp48 == null) {
        downpaymentTaxableAmountSegGrp48 = new INVOIC.SegGrp48();
    }

    downpaymentTaxableAmountSegGrp48.withMOA(
        new MOAMonetaryAmount()
            .withC516(
                new C516MonetaryAmount()
                    .withE5025("165")
                    .withE5004(downpaymentTaxableAmount)
            )
    );
}

if (downpaymentTaxableAmountSegGrp48 != null) {
    invoic.withSegGrp48(downpaymentTaxableAmountSegGrp48);
}

INVOIC.SegGrp48 payableAmountSegGrp48 = null;

BigDecimal payableAmount = invoicJson.getBigDecimal("payableAmount");
if (payableAmount != null) {
    if (payableAmountSegGrp48 == null) {
        payableAmountSegGrp48 = new INVOIC.SegGrp48();
    }

    payableAmountSegGrp48.withMOA(
```

```
        new MOAMonetaryAmount()
            .withC516(
                new C516MonetaryAmount()
                    .withE5025("9")
                    .withE5004(payableAmount)
            )
    );
}

if (payableAmountSegGrp48 != null) {
    invoic.withSegGrp48(payableAmountSegGrp48);
}

INVOIC.SegGrp48 totalVatAmountSegGrp48 = null;

BigDecimal totalVatAmount = invoicJson.getBigDecimal("totalVatAmount");
if (totalVatAmount != null) {
    if (totalVatAmountSegGrp48 == null) {
        totalVatAmountSegGrp48 = new INVOIC.SegGrp48();
    }

    totalVatAmountSegGrp48.withMOA(
        new MOAMonetaryAmount()
            .withC516(
                new C516MonetaryAmount()
                    .withE5025("ZZZ")
                    .withE5004(totalVatAmount)
            )
    );
}

if (totalVatAmountSegGrp48 != null) {
    invoic.withSegGrp48(totalVatAmountSegGrp48);
}

INVOIC.SegGrp50 segGrp50 = null;

String taxTypeCode = invoicJson.getString("taxTypeCode");
if (!isEmpty(taxTypeCode)) {
    if (segGrp50 == null) {
        segGrp50 = new INVOIC.SegGrp50();
    }

    TAXDutyTaxFeeDetails taxDutyTaxFeeDetails = new TAXDutyTaxFeeDetails().withE5283("7");
```

```
C241DutyTaxFeeType c241DutyTaxFeeType = new C241DutyTaxFeeType();
c241DutyTaxFeeType.withE5153(E5153DutyTaxFeeTypeCoded.valueOf(taxTypeCode));

String currentTaxDesc = invoicJson.getString("currentTaxDesc");
if (!isEmpty(currentTaxDesc)) {
    c241DutyTaxFeeType.withE5152(currentTaxDesc);
}

String currentTaxAssessmentBasis = invoicJson.getString("currentTaxAssessmentBasis");
if (!isEmpty(currentTaxAssessmentBasis)) {
    taxDutyTaxFeeDetails.withE5286(currentTaxAssessmentBasis);
}

String currentTaxRate = invoicJson.getString("currentTaxRate");
if (!isEmpty(currentTaxRate)) {
    taxDutyTaxFeeDetails.withC243(
        new C243DutyTaxFeeDetail()
            .withE5278(currentTaxRate)
    );
}

taxDutyTaxFeeDetails.withC241(c241DutyTaxFeeType);
segGrp50.withTAX(taxDutyTaxFeeDetails);
}

BigDecimal addedTaxAmount = invoicJson.getBigDecimal("addedTaxAmount");
if (addedTaxAmount != null) {
    if (segGrp50 == null) {
        segGrp50 = new INVOIC.SegGrp50();
    }

    segGrp50.withMOA(
        new MOAMonetaryAmount()
            .withC516(
                new C516MonetaryAmount()
                    .withE5025("150")
                    .withE5004(addedTaxAmount)
            )
    );
}

BigDecimal otherTaxAmount = invoicJson.getBigDecimal("otherTaxAmount");
if (otherTaxAmount != null) {
    if (segGrp50 == null) {
```

```
        segGrp50 = new INVOIC.SegGrp50();
    }

    segGrp50.withMOA(
        new MOAMonetaryAmount()
            .withC516(
                new C516MonetaryAmount()
                    .withE5025("124")
                    .withE5004(otherTaxAmount)
            )
    );
}

if (segGrp50 != null) {
    invoic.withSegGrp50(segGrp50);
}

INVOIC.SegGrp51 segGrp51 = null;

String totalAllowanceOrCharge = invoicJson.getString("totalAllowanceOrCharge");
if (!isEmpty(totalAllowanceOrCharge)) {
    if (segGrp51 == null) {
        segGrp51 = new INVOIC.SegGrp51();
    }

    segGrp51.withALC(
        new ALCAAllowanceOrCharge()
            .withE5463(E5463AllowanceOrChargeQualifier.fromValue(totalAllowanceOrCharge))
            .withC552(
                new C552AllowanceChargeInformation()
                    .withE1230(totalAllowanceOrCharge)
            )
    );
}

BigDecimal totalDiscountOrChargeAmount = invoicJson.getBigDecimal("totalDiscountOrChargeAmount");
if (totalDiscountOrChargeAmount != null) {
    if (segGrp51 == null) {
        segGrp51 = new INVOIC.SegGrp51();
    }

    segGrp51.withMOA(
        new MOAMonetaryAmount()
            .withC516(
```

```

        new C516MonetaryAmount()
            .withE5025("8")
            .withE5004(totalDiscountOrChargeAmount)
        )
    );
}

if (segGrp51 != null) {
    invoic.withSegGrp51(segGrp51);
}

JAXBElement invoiceEle = new JAXBElement<>(new QName("http://www.ibm.com/edifact/D96A",
"INVOIC", "D96A"), INVOIC.class, invoic);

message.withContent(invoiceEle);

String messageSegments = invoicJson.getString("messageSegments");
String tailMessageReferenceNum = invoicJson.getString("tailMessageReferenceNum");

if (!isEmpty(messageSegments) && !isEmpty(tailMessageReferenceNum)) {
    message.withContent(
        new JAXBElement<>(new QName("UNT"), UNTMessageTrailer.class,
            new UNTMessageTrailer()
                .withE0074(new BigDecimal(Double.valueOf(messageSegments)))
                .withE0062(tailMessageReferenceNum))
    );
}

interchange.withMessage(message);

interchange.withUNZ(
    new UNZInterchangeTrailer()
        .withE0036(new BigDecimal(1))
        .withE0020("17")
);

// Turn Java model into XML
ByteArrayOutputStream byteArrayOutputStream = new ByteArrayOutputStream();
//JAXBContext jaxbContext = JAXBContext.newInstance(Interchange.class,
org.smooks.edifact.binding.service.ObjectFactory.class, org.smooks.edifact.binding.d03b.ObjectFactory.class);
//jaxbContext.createMarshaller().marshal(interchange, byteArrayOutputStream);

//${request.body} = new String(byteArrayOutputStream.toByteArray());

```

```

SmooksCaches.marshal(
    interchange,
    byteArrayOutputStream,
    Interchange.class,
    org.smooks.edifact.binding.service.ObjectFactory.class,
    org.smooks.edifact.binding.d96a.ObjectFactory.class
);

${request.body} = SmooksCaches.translate("d96a", "INVOIC", byteArrayOutputStream,
SmooksCaches.class.getClassLoader());

```

注： 自定义脚本中展示了将 JSON 转为 EDIFACT 的完整流程，通常需要用户先确定入参类型(如：JSON、XML等)，再根据合作方的 EDIFACT 字段要求进行编写转换逻辑，示例脚本并不通用；该部分需要操作人员一定编程能力，同时了解 EDIFACT 规范！

注： 自定义脚本中依赖类运行时已全部导入，如果要在编辑器中调试，需手动添加相关依赖jar包到classpath中。

1. 程序说明

```

// 依赖部分省略

public static boolean isEmpty(String val) {
    return val == null || val.equals("");
}

// 程序步骤1：从 exchange 中获取 json 格式的发票请求并转为 JSONObject
String invoicJsonStr = exchange.getMessage().getBody(String.class);
JSONObject invoicJson = JSON.parseObject(invoicJsonStr);

// 程序步骤2：定义 EDIFACT Java 实体，获取 json 中字段并配置
Interchange interchange = new Interchange();

// UNA
interchange.withUNA(
    new UNA()
        .withCompositeSeparator(":")
        .withFieldSeparator("+")
        .withDecimalSeparator(".")
        .withEscapeCharacter("?")
        .withRepeatSeparator("*")
        .withSegmentTerminator("")
);

// UNB ... 省略

```



```

Message message = new Message();

// ... message配置部分省略

interchange.withMessage(message);

interchange.withUNZ(
    new UNZInterchangeTrailer()
        .withE0036(new BigDecimal(1))
        .withE0020("17")
);

// 程序步骤3: 定义输出流, 用于存放由 Java EDIFACT 转为 XML 的流数据
ByteArrayOutputStream byteArrayOutputStream = new ByteArrayOutputStream();

// 程序步骤4: 将 interchange 转为 XML 并写入 byteArrayOutputStream
// 注: SmooksCaches 会缓存 XML 相关转换上下文, 为后续转换操作提速
SmooksCaches.marshal(
    interchange,
    byteArrayOutputStream,
    Interchange.class,
    org.smooks.edifact.binding.service.ObjectFactory.class,
    org.smooks.edifact.binding.d96a.ObjectFactory.class
);

// 程序步骤4: 将 byteArrayOutputStream 中的 XML 转换为 EDI, 并设置到 exchange 中
${request.body} = SmooksCaches.translate("d96a", "INVOIC", byteArrayOutputStream,
SmooksCaches.class.getClassLoader());

```

2. 主要类说明

- `org.smooks.edifact.binding.{edifact-version}.Interchange`

`Interchange` 为 EDIFACT 主类, 该类是通过官方定义生成的标准 Java 类; 其中 `{edifact-version}` 为指定的 EDIFACT 版本, 比如示例中的 `d96a`。

`Interchange` 实例中必须配置 `UNA`、`UNB`、`Message`、`UNZ`, 具体配置需要根据用户的数据定义、**EDIFACT** 官方定义以及用户的数据要求来进行编程配置。

- `ByteArrayOutputStream`

`Interchange` 实例转为 XML 后的输出流, 用于后续转换操作。

- `SmooksCaches`

`SmooksCaches` 主要提供 `marshal` 和 `translate` 方法; `marshal` 主要初始化转换过程的上下文, 同时缓存该上下文供后续转换请求复用; `translate` 方法用于将 `Interchange` 实例输出的 XML 转为标准

EDIFACT。

